

Матюхін Олександр дип антиплагіат.docx

 Yuriy Fedkovych Chernivtsi National University

Document Details

Submission ID

trn:oid::2945:391120422

Submission Date

Jun 4, 2026, 10:18 AM GMT+3

Download Date

Jun 4, 2026, 10:34 AM GMT+3

File Name

Матюхін Олександр дип антиплагіат.docx

File Size

4.8 MB

90 Pages

12,571 Words

91,475 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Cited Text
- Small Matches (less than 10 words)
- Crossref database

Top Sources

- 2%  Internet sources
- 0%  Publications
- 2%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Top Sources

- 2% Internet sources
- 0% Publications
- 2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	University of Bristol on 2015-04-24	<1%
2	Internet	pdffox.com	<1%
3	Internet	fractalswithr.blogspot.com	<1%
4	Internet	uk.wikipedia.org	<1%
5	Internet	dspace.op.edu.ua	<1%
6	Internet	geoinfotheory.org	<1%
7	Submitted works	Manchester Metropolitan University on 2015-04-21	<1%
8	Internet	nure.ua	<1%
9	Submitted works	The Art Institute Online on 2013-10-02	<1%
10	Submitted works	West University Of Timisoara on 2019-06-26	<1%
11	Internet	coek.info	<1%

12	Internet	arxiv.org	<1%
13	Submitted works	Jamnabai Narsee School on 2024-12-12	<1%
14	Internet	kipdf.com	<1%
15	Submitted works	Vinnytsia Technical Applied College on 2026-01-30	<1%
16	Internet	eftj.secna.ru	<1%
17	Internet	eprints.cdu.edu.ua	<1%
18	Internet	hdl.handle.net	<1%
19	Internet	lijianping.cn	<1%
20	Internet	ww2.ii.uj.edu.pl	<1%

ВСТУП

Актуальність теми. Стрімка цифровізація економіки й суспільства, поширення хмарних сервісів, мобільних застосунків, IoT-пристроїв та розподілених систем зумовлюють постійне зростання обсягу даних, що передаються та зберігаються у відкритих або напіввідкритих середовищах. З іншого боку, активного розвитку набули технології квантового обчислення та інструменти штучного інтелекту, які можуть значно спростити та пришвидшити процес криптоаналізу. Зокрема, алгоритм Шора [1], розроблений для зламу криптосистем за допомогою квантових комп'ютерів, за умови повної реалізації квантового обчислення скорочує час дешифрування практично стійких шифрів із тисячоліть до лічених хвилин.

За таких умов ключовим чинником довіри до інформаційних систем стає забезпечення високого рівня криптографічного захисту, а саме: забезпечення принципів конфіденційності, цілісності, автентичності та незаперечності дій [2]. Ці принципи в сучасній криптографії спираються на наявність якісного криптографічного інструментарію: надійних генераторів псевдовипадкових послідовностей, стійких ключових потоків, випадкових підстановок і перестановок. Дефіцит або низька якість випадковості є однією з найчастіших причин провалів у реальних системах захисту: навіть сильний шифр може втратити стійкість через слабкий матеріал ключа чи передбачуваний генератор.

У зв'язку з цим перед криптографами постало завдання розробки алгоритмів із максимально хаотичним розподілом бітів шифртексту за умови повної відтворюваності даних. Що викликало зростання інтересу до альтернативних джерел криптографічної псевдовипадковості, зокрема до детермінованих хаотичних систем. Детермінований хаос поєднує дві на перший погляд суперечливі властивості: строгий причинно-наслідковий опис через диференціальні рівняння або дискретні відображення та водночас високу чутливість до початкових умов і параметрів, що на практиці проявляється як

складна, «шумоподібна» поведінка, що виглядає як хаос для стороннього спостерігача [3]. Саме ці риси роблять детермінований хаос привабливим для криптографії: малі зміни ключа можуть призводити до радикально інших траєкторій, а вихідні послідовності можуть демонструвати широкосмугові спектральні властивості та високі статистичні показники «випадковості».

Поняття хаотичної динаміки сформувалося у другій половині XX століття на стику математики, фізики та теорії систем. Однією з відправних точок вважаються роботи Edward Lorenz [4], у яких було продемонстровано, що прості детерміновані рівняння можуть породжувати складні неперіодичні траєкторії та явище чутливості до початкових умов. Згодом спектр досліджуваних хаотичних моделей розширився: з'явилися інші низьковимірні системи та інструменти кількісного аналізу хаосу [5].

Важливим аспектом теорії хаосу є використання апарату фракталів для дослідження динаміки систем. Це зумовлено тим, що дивний атрактор – ключова характеристика хаотичної системи – не може бути представлений у вигляді простої кривої чи поверхні, а має складну фрактальну структуру. Він визначається як підмножина фазового простору, до якої з плином часу прямують усі траєкторії, що стартують не надто далеко від неї. При цьому рух на атракторі є нестійким: дві траєкторії, випущені з близьких точок, з часом значно розбігаються. Ця властивість накладає фундаментальні обмеження на прогнозування: для лінійного збільшення часу достовірного прогнозу точність початкових даних необхідно підвищувати експоненціально. Це означає, що навіть за використання потужних комп'ютерів неможливо з прийнятною точністю розрахувати траєкторію на тривалий період. На кожному кроці обчислень неминуче вносяться похибки (через округлення чисел і специфіку чисельних методів), які швидко накопичуються, внаслідок чого знайдена траєкторія починає істотно відрізнятися від справжньої. Таке спотворення неможливо виправити простим збільшенням обчислювальної потужності, саме це і означає динамічний хаос [3].

Перенесення ідей хаотичної динаміки у криптографію набуло особливої активності наприкінці ХХ – на початку ХХІ століття. Дослідники запропонували хаос-орієнтовані шифри на основі дискретних відображень (логістичне, кубічне та інші) та безперервних осциляторів (з вимушуванням і дисипацією), а також схеми шифрування з використанням синхронізації хаотичних систем при передачі даних [6]. У межах цього напрямку важливе місце займають підходи, де хаотична траєкторія перетворюється на ключовий потік або керує перестановками/підстановками у процесі шифрування.

Таким чином, сучасні дослідження демонструють перспективність використання хаотичних систем у криптографії. Однак для практичного застосування ключовими є перевірка статистичних властивостей випадковості згенерованих послідовностей та порівняльний аналіз запропонованих схем, що визначає подальшу логіку цієї роботи.

Об'єктом дослідження є хаотичні системи як математична основа побудови криптографічних генераторів псевдовипадкових послідовностей.

Предметом дослідження є алгоритми шифрування, реалізовані на основі об'єднання дискретного Логістичного відображення з такими неперервними динамічними детермінованими нелінійними системами, як Лоренца, Ресслера, Чуа, осцилятор Дуффінга, осцилятора Ван дер Поля та маятник з силою, що змушує (Forced Pendulum).

Метою дослідження є здійснення порівняльного аналізу систем хаосу, розробка шифраторів на основі детермінованих систем хаосу та демонстраційного програмного забезпечення з функціоналом шифрування та розшифрування даних, тестування ефективності розроблених шифрів.

Завдання у відповідності з метою дослідження:

1) визначити параметричні області, де спостерігається стійкий хаотичний режим досліджуваних систем та провести порівняльний аналіз характеристик динаміки хаосу в базових системах, виділивши потенційно перспективні системи придатні для генерації криптографічних послідовностей;

2) сформувати гібридні комбінації детермінованих хаотичних систем;

- 3) розробити гібридний спосіб перетворення станів хаотичної системи у числовий/байтовий потік із забезпеченням відтворюваності;
- 4) змодельовати алгоритми генераторів псевдовипадкових послідовностей та шифрування;
- 5) розробити шифри на основі генераторів псевдовипадкових послідовностей;
- 6) провести аналіз криптостійкості розроблених генераторів за допомогою пакету тестів NIST-STS;
- 7) розробити програмне забезпечення для демонстрації шифрування та розшифрування тексту, зображень, аудіо та файлів;
- 8) сформулювати висновки щодо практичної застосовності підходу, визначити обмеження, ризики реалізації.

Методи дослідження, застосовані у кваліфікаційній роботі, включають аналіз літературних джерел, використання інтелектуального пошуку з подальшим порівняльним аналізом ступеня хаотичності досліджуваних систем, статистичний аналіз згенерованих послідовностей за допомогою програмного забезпечення NIST Statistical Test Suite (NIST-STS).

Очікуваний внесок і практичне значення: використано набір методичних підходів, зокрема порівняльний аналіз детермінованих хаотичних систем, гібридну стратегію комбінування систем хаосу, на базі яких розроблено принцип формування генераторів псевдовипадкової послідовності. Практичне значення полягає у створенні генераторів псевдовипадкових послідовностей, які можуть бути використані для подальшого розвитку гібридних генераторів і шифрувальних модулів.

Пояснювальна записка до випускної роботи містить 105 сторінок, 41 ілюстрацію та 9 таблиць, список літературних джерел містить 33 найменування.

РОЗДІЛ 1. ДОСЛІДЖЕННЯ ШИФРІВ НА ОСНОВІ СИСТЕМ ХАОСУ

1.1. Концептуальні засади хаотичної криптографії

Відомо, що для хаотичних динамічних систем характерні висока складність та непередбачуваність. Ці властивості лежать в основі практичної цінності детермінованого хаосу для криптографії. Даний факт обумовлений тим, що як і в теорії хаосу, так і в криптографії малі зміни початкових параметрів викликають радикальні зміни всієї подальшої еволюції системи [7].

Якщо розглядати це питання ширше, то для хаотичної системи характерно, що вона функціонує в обмеженому просторі станів, проте її траєкторії швидко розходяться навіть за незначних змін початкового стану і з часом охоплюють весь допустимий фазовий простір. Така хаотична поведінка обумовлена наявністю певних властивостей, серед яких ключовими є топологічна транзитивність та висока чутливість до початкових умов.

З іншого боку, для запобігання появи структурних закономірностей та статистичних залежностей, які можуть бути використані при криптоаналізі, потрібна здатність шифру залишатися в межах допустимого простору подання даних з одночасним забезпеченням рівномірного заповнення простору можливих шифртекстів. Це і є пряма аналогія топологічної транзитивності. Тому такі вимоги, аналогічні до топологічної транзитивності, висуваються і до криптографічних систем.

Одночасно з цим, незначна зміна вхідних даних має призводити до суттєвих змін результату шифрування, що забезпечує високий рівень дифузії та стійкість до аналізу. Тобто чутливість криптографічного алгоритму до відкритого тексту та ключа відповідає чутливості хаотичної системи до початкових умов.

Таким чином, між криптографією та детермінованим хаосом існує концептуальна схожість: в обох випадках інформація піддається нелінійному перетворенню, яке є строго детермінованим, але при цьому практично непередбачуваним для зовнішнього спостерігача. Практична цінність цього

факту підтверджується тим, що протягом останніх років спостерігається значне зростання інтересу до використання хаотичних динамічних систем у криптографії.

1.2. Порівняльний аналіз динамічних характеристик базових хаотичних систем

Динамічні детерміновані системи характеризуються переліком параметрів, серед яких основними для опису хаосу є швидкість розбіжності траєкторій, фрактальна розмірність, ентропія, частотний спектр, вікна періодичності та точки біфуркацій.

Швидкість розбіжності траєкторій характеризується таким показником, як спектр Ляпуновських показників:

$$\{\lambda_1, \lambda_2, \dots, \lambda_n\}$$

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$$

$$\lambda_i = \lim_{t \rightarrow \infty} \frac{1}{t} \ln |v_i(t)|$$

Де:

λ_i – i -й показник Ляпунова.

$v_i(t)$ – власні значення (singular values) лінійної матриці еволюції (матриці Якобі) вздовж траєкторії.

t – час (для неперервних систем) або кількість ітерацій (для дискретних відображень) [8].

Фрактальна розмірність дивного атрактора динамічного хаосу характеризується таким показником як формула Каплана-Йорка:

$$D_{KY} = k + \frac{\sum_{i=1}^k \lambda_i}{|\lambda_{k+1}|}$$

Де k — найбільше ціле число, для якого сума перших k показників Ляпунова ще залишається додатною ($\sum_{i=1}^k \lambda_i \geq 0$) [3, с. 127].

Ентропія детермінованого хаосу характеризується таким показником як формула Коломогорова-Сіная:

$$h_{KS} = \sum_{\lambda_i > 0} \lambda_i$$

$$\{\lambda_1, \lambda_2, \dots, \lambda_n\}$$

Оскільки вікна періодичності характеризуються порядком Шарковського виключно в одновимірних динамічних системах, їх визначення в двох та трьох вимірних системах ускладнене [9].

Таким чином, базовими показниками для порівняльного аналізу динамічних систем хаосу в даному дослідженні використовуються [10]:

- спектр Ляпуновських показників (спектр ЛП);
- формула Каплана-Йорка;
- формула Коломогорова-Сіная;
- частотний спектр (спектр Фур'є).

Досліджуваними системами є:

- логістичне відображення;
- система Лоренца;
- система Ресслера;
- система Чуа;
- осцилятора Дуффінга;
- осцилятора Ван дер Поля;

– маятник з силою, що змушує (Forced Pendulum).

Логістичне відображення – це поліноміальне відображення, що описує зміну чисельності популяції з плином часу та є прикладом простих нелінійних рівнянь зі складною, хаотичною поведінкою. Система є дискретною, оскільки відображає приріст популяції в дискретні моменти часу.[3, с. 198]

Система Лоренца – це система трьох нелінійних диференціальних рівнянь, яку вперше запропоновано як спрощену модель атмосферної конвекції. За певних значень параметрів її траєкторії в фазовому просторі залишаються обмеженими, проте демонструють хаотичну, неперіодичну динаміку (зокрема чутливість до початкових умов). У результаті в фазовому просторі формується дивний атрактор – геометрична структура, що відображає хаотичний рух системи всередині обмеженої області.

Система Ресслера – це тривимірна система звичайних диференціальних рівнянь, запроваджена для опису хаотичної поведінки в хімічних реакціях. Вона демонструє дивний атрактор, що має фрактальну структуру, та описує перехід від періодичних коливань до хаосу.

Система Чуа – найпростіший електричний ланцюг, що демонструє режими хаотичних коливань та є канонічним прикладом для дослідження хаотичних режимів. Система складається з двох конденсаторів, однієї котушки індуктивності, лінійного резистора та нелінійного резистора з негативним опором.

Осцилятор Дуффінга – це фундаментальна нелінійна динамічна система з відновлювальною силою, що містить кубічну нелінійність, часто з урахуванням демпфування та періодичного зовнішнього збурення. Вона є одним із класичних прикладів систем, здатних переходити до хаотичного руху за певних параметрів зовнішньої сили та демпфування.

Класичний нелінійний осцилятор Ван дер Поля – це приклад динамічної системи з граничним циклом Пуанкаре. Таким чином, класична форма осцилятора Ван дер Поля не демонструє хаотичної динаміки. Тому потрібно використовувати його модифікації (вимушені форми), які призводять до

детермінованого хаосу. Наприклад, шляхом додавання до системи зовнішнього періодичного впливу та/або кубічної нелінійності.

Маятник з силою, що змушує (forced pendulum) — рух маятника під впливом зовнішньої періодичної сили. На відміну від вільних коливань, які згодом згасають через тертя, вимушені коливання дозволяють підтримувати рух чи навіть приводити систему до хаосу. При певних значеннях амплітуди та частоти зовнішньої сили рух маятника стає непередбачуваним (хаотичним), хоча система залишається детермінованою.

1.2.1 Порівняльна характеристика динаміки хаосу та обчислювальної складності базових нелінійних систем

З метою проведення порівняльного аналізу характеристик систем за хаотичністю та складністю здійснено інтелектуальний пошук і аналіз літературних джерел, у яких наведено сталі значення параметрів хаосу та відповідні кількісні показники хаотичності. Із опрацьованих джерел отримано параметри хаосу для систем логістичного відображення, Лоренца, Ресслера, Чуа, осцилятора Дуффінга, осцилятора Ван дер Поля та маятника з силою, що змушує.

Для кожної з розглянутих систем за відповідних параметрів було визначено спектр показників Ляпунова, який використовується як основна кількісна характеристика хаотичності динамічної системи. Зокрема, було визначено показник Каплана-Йорка для оцінки фрактальної розмірності атрактора, а також ентропію Колмогорова-Сіная, що характеризує ступінь непередбачуваності динаміки системи. Додатково залучається спектральний аналіз Фур'є для якісного визначення хаотичності на основі аналізу періодичності станів системи.

Отримані дані було зведено до табличного формату (Додаток А).

На основі зведеної таблиці було сформовано ієрархію показників хаосу: інтенсивність хаосу оцінено за найбільшим показником Ляпунова λ_1 , а структурну складність атрактора — за фрактальною розмірністю Каплана-Йорка

D_{KY} . Основний порядок визначено за h_{KS} та λ_1 ; D_{KY} використано як допоміжний аргумент у поясненнях.

1. Система Лоренца має найвищу інтенсивність хаосу: $\lambda_1 \approx 0,90557$ ($h_{KS} \approx 0,90557$), що означає найшвидше експоненціальне розходження траєкторій. Попри помірну $D_{KY} \approx 2,062$, система демонструє найбільш «жорсткий» хаос за темпом втрати передбачуваності.

2. Логістичне відображення для класичного хаотичного режиму має додатний показник Ляпунова, який дорівнює $\lambda = \ln 2 \approx 0,6931$. Це робить відображення другим за інтенсивністю хаосу після Лоренца.

3. Система Чуа має високу інтенсивність хаосу: $\lambda_1 \approx 0,432495$ ($h_{KS} \approx 0,432495$). Додатково має одну з найбільших фрактальних розмірностей серед неперервних 3D-систем у таблиці: $D_{KY} \approx 2,1021$, що відображає відносно складну геометрію атрактора.

4. Дуффінга (Duffing, forced) – Помірна інтенсивність хаосу за $\lambda_1 \approx 0,1416$ ($h_{KS} \approx 0,1416$), але найбільша $D_{KY} \approx 2,4146$ серед наведених неперервних систем. Це означає: хаос «повільніший» за темпом розходження траєкторій, зате атрактор структурно складніший (вища ефективна розмірність).

5. Forced Pendulum (damped driven) – $\lambda_1 \approx 0,129834$ ($h_{KS} \approx 0,129834$) - хаос слабший, ніж у Дуффінга, але має відносно високу фрактальність $D_{KY} \approx 2,2061$. Характеризується широкосмуговим спектром, типовим для хаотичних коливань у вимушеному маятнику.

6. Ван дер Поля (forced, хаос) – $\lambda_1 \approx 0,109414$ ($h_{KS} \approx 0,109414$) - ще нижча інтенсивність хаосу. $D_{KY} \approx 2,0453$ близька до 2, що відповідає менш «об'ємній» геометрії атрактора порівняно з маятником і Чуа.

7. Система Ресслера – найменша інтенсивність хаосу серед наведених систем: $\lambda_1 \approx 0,07135$ ($h_{KS} \approx 0,07135$) та $D_{KY} \approx 2,01323$. За цими метриками в таблиці Ресслер має найменшу «глибину хаосу» (повільніший темп розбігання та майже двовимірна ефективна структура атрактора).

Наступним етапом порівняльного аналізу є визначення обчислювальної складності систем хаосу за типом системи (дискретна, неперервна), кількістю змінних, кількістю операцій на обчислення та оцінкою ресурсоемності. Отримані дані було зведено до табличного формату (Додаток А).

На основі аналізу глибини хаосу та обчислювальної складності було сформовано ієрархію систем, що відображає їх відносну придатність і перспективність для застосування в криптографічних задачах.

Логістична система є найкращою в переліку моделей. Враховуючи високий показник хаосу $\lambda_1 \approx 0.693$ (другий за величиною після Лоренца), вона забезпечує максимальну обчислювальну простоту, бо потребує лише 3 арифметичні операції на крок. Це робить систему найефективнішою для швидкого отримання псевдовипадкових чисел.

Другою за пріоритетністю можна вважати систему Лоренца. Серед розглянутих моделей вона має найвищу інтенсивність хаосу $\lambda_1 \approx 0.90557$. При цьому за співвідношенням “хаос/обчислювальна складність” вона поступається логістичному відображенню, бо вона є неперервною диференційною системою: хоч її праві частини відносно прості, для моделювання потрібно виконувати чисельне інтегрування Рунге-Кутта, що підвищує обчислювальне навантаження.

Третьою за пріоритетністю є система Чуа, яка демонструє доволі високу інтенсивність хаотичної динаміки за найбільшим показником Ляпунова $\lambda_1 \approx 0.432495$ у поєднанні з підвищеною фрактальною складністю атрактора $D_{ky} \approx 2.1021$. Водночас її обчислювальна складність має середній рівень через наявність кусково-лінійної нелінійності та операцій модуля, що складніше за базові арифметичні операції. Проте це є менш затратно, ніж тригонометричні обчислення. Тому система забезпечує збалансоване поєднання глибини хаосу та високої фрактальності, однак поступається системі Лоренца як за величиною λ_1 , так і за обчислювальною складністю.

Четвертим за пріоритетністю є осцилятор Дуффінга, який характеризується помірною інтенсивністю хаотичної динаміки та демонструє найвищу фрактальну складність серед розглянутих моделей $D_{ky} \approx 2.4146$. При

цьому його програмна реалізація є обчислювально затратною через необхідність врахування розрахунку зовнішньої періодичної сили.

Наступним за пріоритетністю є осцилятор Ван дер Поля, який характеризується значно нижчими значеннями як фрактальної складності атрактора, так і інтенсивності хаотичної динаміки $\lambda_1 \approx 0.1094$. Водночас його обчислювальна складність є близькою до системи Дуффінга: на кожному етапі чисельного інтегрування методу Рунге-Кутта необхідно обчислювати зовнішнє збурення. Тому з точки зору криптографічної реалізації така модель є малоефективною: вона потребує значних обчислювальних витрат, водночас формуючи відносно слабку хаотичну динаміку.

Шостою за списком пріоритетності є система Forced Pendulum. Дана система є найгіршою з точки зору обчислювальної складності, бо у неї присутні одразу дві тригонометричні функції: $\sin$ та $\cos$. При цьому має трохи вищі показники хаосу, ніж у Ван дер Поля, але значно нижчі ніж у осцилятора Дуффінга. Тому дана система за ефективністю для криптографії посідає передостаннє місце, оскільки є найскладнішою для процесора, а рівень хаосу майже в 7 разів нижчий, ніж у системи Лоренца.

Найнижчу ефективність показує система Ресслера. Серед розглянутих моделей вона характеризується найменшою інтенсивністю хаотичної динаміки $\lambda_1 \approx 0.07135$ та найнижчим значенням фрактальної оцінки атрактора $D_{ky} \approx 2.01323$. Водночас за обчислювальною швидкістю вона є близькою до системи Лоренца. Таким чином, за однакової обчислювальної складності система Ресслера забезпечує найгірший рівень хаотичності серед досліджуваних систем, що й зумовлює її найменшу ефективність.

1.2.2 Класифікація досліджуваних систем у контексті змішаної динаміки

Загальний клас систем зі змішаною динамікою охоплює широкий спектр математичних моделей, де змішування відбувається шляхом поєднання різних

структурних компонентів, типів сигналів або законів еволюції в часі. Залежно від природи взаємодії складників, можна виділити наступні ключові категорії:

- гібридні динамічні системи – тип систем, архітектура яких базується на взаємодії дискретних компонентів із неперервними динамічними процесами, що описуються диференціальними рівняннями [22];

- системи зі змішаними модами коливань – складні неперервні системи, що демонструють поєднання різних типів динамічної поведінки (наприклад, чергування коливань малої амплітуди з великими спалахами/піками). Вони формуються шляхом поєднання кількох неперервних підсистем або через специфічну структуру фазового простору [23];

- просторово-часові змішані системи – системи, в яких змішування простих динамічних систем відбувається не лише в часі, а й у просторі. Це дозволяє генерувати складну просторову динаміку на основі взаємодії елементарних хаотичних вузлів [24];

Згідно з наведеними типами змішування, системи, що розглядаються у даній роботі, належать до класу гібридних динамічних систем. Це зумовлено тим, що вони базуються на поєднанні дискретного логістичного відображення, яке виконує роль керуючого механізму або додаткового джерела хаотичності, та класичних неперервних детермінованих систем, таких як системи Лоренца, Чуа, Дуффінга та інших. Така взаємодія систем дозволяє використовувати переваги обох типів динаміки для підвищення криптографічної стійкості генерованих послідовностей.

1.3. Огляд традиційних генераторів бінарної ключової послідовності (ГБКП) та способів їх застосування

Послідовності в криптографії можна розділити на два класи: випадкові та псевдовипадкові. Випадкові послідовності, як правило, породжуються апаратними або комбінованими, програмно-апаратними засобами, які використовують датчики фізичних величин як генеруючий пристрій. Але

2 оскільки для реалізації дійсно випадкових послідовностей зазвичай використовується дороге вибагливе обладнання, для повсякденного використання застосовують генератор псевдовипадкових чисел. Відмінності між істинно випадковими та псевдовипадковими послідовностями дуже значні. Основна відмінність, полягає в тому, що псевдовипадкова послідовність періодична. Іншою важливою відмінністю є те, що за однакових початкових умов псевдовипадкова послідовність буде однаковою [25].

4 У зв'язку з цими особливостями, генератор псевдовипадкової послідовності реалізований як алгоритм або пристрій, який за допомогою детермінованої процедури та початкового значення породжує послідовність чисел, що за своїми статистичними властивостями наближається до послідовності істинно випадкових чисел [26].

Завдяки доступності та достатньому рівню надійності генератори псевдовипадкових послідовностей широко застосовуються в криптографії, зокрема для:

- формування ключів симетричних і асиметричних криптосистем;
- створення цифрових підписів;
- реалізації криптографічних протоколів;
- криптографічної аутентифікації;
- потокового шифрування;
- інших криптографічних задач

Серед таких генераторів псевдовипадкових послідовностей відомими є алгоритм RSA та алгоритм Блума–Мікалі.

1.3.1 Опис генератора Блум–Мікалі

Генератор Блума–Мікалі, запропонований Мануелем Блумом та Сільвіо Мікалі, є криптографічно стійким генератором псевдовипадкових бітів. Безпека генератора ґрунтується на складності обчислення квадратичного кореня по

модулю великого простого числа, що робить відновлення вихідної послідовності практично неможливим без знання секретного ключа [27].

Принцип роботи:

1. Вибирається параметра p , як велике просте числа та примітивний корінь g за модулем p .
2. Вибирається випадкове початкове значення x_0 , яке відповідає умові $1 \leq x_0 < p$ та не ділиться на p .
3. Обчислюється новий стан на кожному кроці:

$$x_{i+1} = g^{x_i} \bmod p$$

4. Формується вихідна послідовність з поточного значення x_i .
5. Процес багаторазово повторюється до отримання псевдовипадкової послідовності необхідної довжини.
6. Безпека генератора обумовлюється його криптографічною непередбачуваністю, тобто жоден поліноміальний за часом алгоритм не може передбачити майбутні біти послідовності, якщо відомі лише попередні біти. Навіть якщо зломисник знає частину послідовності або попередні стани, передбачити значення наступних бітів без знання початкового значення x_0 є практично неможливим. Це забезпечується тим, що обчислення дискретного логарифму за модулем великого простого числа вважається нерозв'язною задачею в практичних умовах.

1.3.2 Опис генератора RSA

RSA-генератор псевдовипадкових чисел – це криптографічно стійкий генератор, заснований на асиметричному алгоритмі RSA, який використовується для формування псевдовипадкової послідовності. Його стійкість спирається на обчислювальну складність завдання факторизації великих чисел, а

послідовність, що отримується, має властивості статистичної і криптографічної випадковості [28].

Принцип роботи:

1. Аналогічно криптосистемі RSA вибирається велике складне число $N = p \cdot q$, де p і q два випадкових великих простих числа.
2. Далі вибирається експонента e , яка є взаємно простою з функцією Ейлера $\varphi(N)$.
3. Випадковим чином генерується число x_0 , яке менше за N і взаємно просте з ним.
4. На кожному кроці генерації обчислюється рекурентне співвідношення:

$$x_{i+1} = x_i^e \bmod N$$

5. З отриманого значення витягуються певні біти, які формують частину наступної псевдовипадкової послідовності.

6. Процес повторюється багаторазово до утворення послідовності біт необхідної довжини.

Безпека даної системи забезпечується обчислювальною складністю факторизації великого числа N . За умови, що завдання факторизації залишається обчислювально нерозв'язним, передбачення наступного елемента послідовності на основі попередніх значень вважається практично неможливим.

1.3.3 Аналіз досліджуваних методів генерації псевдовипадкових послідовностей

1. Генератор Блума-Мікалі [29]

Переваги:

— математично доведена стійкість: безпека методу ґрунтується на припущенні, що задача дискретного логарифма належить до класу NP-складних, що робить передбачення наступного біта практично неможливим;

- висока якість вихідної послідовності: псевдовипадковість результатів має суворе теоретичне обґрунтування;

- архітектурна лаконічність: алгоритм має просту логіку, яка базується на циклічних ітераціях та чіткому правилі отримання біта;

Недоліки:

- низька продуктивність: оскільки для отримання кожного біта необхідно виконувати операцію піднесення до степеня з великими числами, метод є доволі повільним;

- обмежена пропускна здатність: генерація лише одного біта за одну ітерацію робить цей підхід неефективним для опрацювання великих обсягів даних;

- складність реалізації на апаратному рівні: операції з великими числами потребують значних потужностей процесора та споживають багато енергії.

- проблема генерації параметрів: необхідність пошуку великих безпечних простих чисел для модуля, що є ресурсомістким підготовчим етапом.

2. Генератор RSA [30]

Переваги:

- технологічна сумісність: легко інтегрується в сучасні системи, оскільки підтримка RSA наявна в більшості криптографічних бібліотек;

- можливість оптимізації: за умови дотримання певних критеріїв безпеки, алгоритм дозволяє витягувати декілька бітів за одну ітерацію, що дещо підвищує його швидкість.

- гнучкість рівнів безпеки: можна легко масштабувати стійкість генератора, просто збільшуючи довжину ключа (наприклад, з 2048 до 4096 біт), не змінюючи архітектуру самого алгоритму.

Недоліки:

- чутливість до конфігурації: стійкість системи критично залежить від вибору параметрів;

- часові витрати:, RSA не підходить для генерації гігабайтів даних у режимі реального часу через значні обчислювальні витрати;

- ризики при збільшенні кількості бітів: спроба пришвидшити роботу шляхом виведення більшої кількості бітів потребує ретельного обґрунтування, оскільки невдала реалізація може суттєво послабити криптографічну стійкість.

За результатами аналізу можна зробити такий висновок, що генератори, Blum–Micali та RSA забезпечують високий рівень криптографічної стійкості, оскільки базуються на математичних проблемах факторизації та дискретному логарифмуванні. Проте їхнє практичне використання стримується низькою швидкістю та високими вимогами до ресурсів. Попри це, RSA виглядає дещо практичнішим завдяки можливості отримувати кілька бітів за одну ітерацію та широкій підтримці в криптографічних бібліотеках, хоча його надійність істотно залежить від правильного добору параметрів. Водночас обидва методи поступово втрачають актуальність, бо в перспективі можуть стати вразливими на тлі розвитку квантових обчислень.

1.4. Методологія статистичного аналізу криптостійкості за допомогою пакету NIST STS

Мінімальна статистична відмінність між випадковими послідовностями та результатами шифрування є однією з ключових вимог до криптографічного програмного забезпечення. Для оцінки ступеня випадковості спеціалістами NIST було розроблено набір статистичних тестів для аналізу генераторів послідовностей. Даний тестовий пакет став стандартом для оцінювання статистичних характеристик шифрів, генераторів та інших криптографічних примітивів і широко використовується криптографічною спільнотою [31].

NIST Statistical Test Suite (NIST-STS) – це набір із статистичних тестів, призначених для аналізу випадковості бінарних послідовностей довільної довжини, що формуються генераторами випадкових та псевдовипадкових чисел.

Дані тести спрямовані на виявлення різних форм статистичної невинадковості, які можуть проявлятися у досліджуваних послідовностях.

Пакет складається з 15 статистичних тестів:

1. Перевірка рівномірності розподілу одиниць у блоці
2. Перевірка максимальних відхилень накопичених сум
3. Перевірка кількості серій у послідовності
4. Перевірка максимальної довжини серії одиниць у блоці
5. Перевірка лінійної незалежності бітів за допомогою рангу матриць
6. Виявлення періодичних складових у двійковій послідовності
7. Перевірка частоти появи перекривних m -бітних шаблонів
8. Оцінка можливості стиснення послідовності
9. Перевірка ентропійних властивостей джерела
10. Перевірка статистичних властивостей випадкового блукання
11. Частота відвідування станів при випадковому блуканні.
12. Перевірка рівномірності розподілу m -бітних підпослідовностей
13. Перевірка складності послідовності за алгоритмом Лемпеля—Зіва
14. Перевірка частоти появи неперекривних шаблонів
15. Перевірка лінійної складності

У роботі використовується графічний інтерфейс NIST Statistical Test Suite 2.1.2 (рис. 1.1), завантажений через NIST_UI.exe.

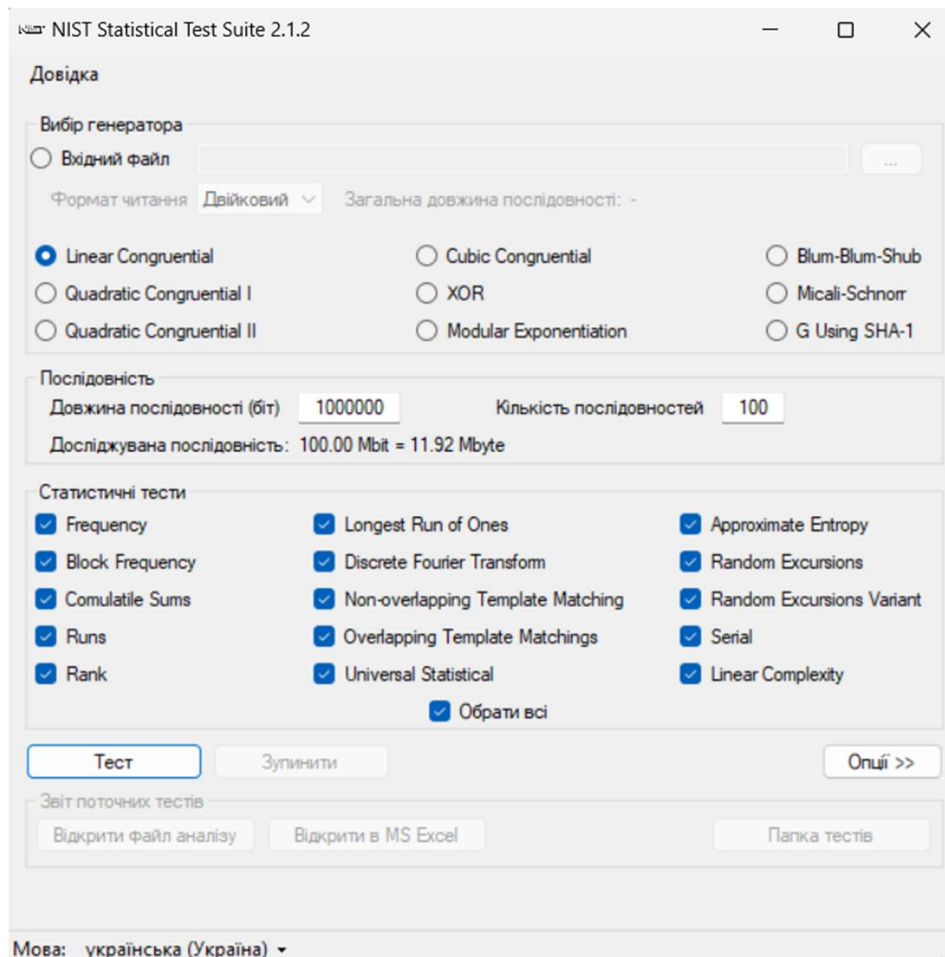


Рисунок 1.1 – Головна форма програми NIST Statistical Test Suite 2.1.2

Після його запуску на екрані з'явиться головна форма пакету, яка дозволяє керувати усіма його можливостями.

Для роботи потрібно попередньо згенерувати файл з послідовністю довжиною 100 млн біт та ввести шлях до файлу. Після цього встановити формат читання у значення «Текстовий». Якщо файл відповідає вимогам, рядок із довжиною файлу підсвітиться зеленим кольором (рис. 1.2); у разі невідповідності – червоним. У блоці «Статистичні тести» можна вибрати набір тестів для виконання. Рекомендується обрати всі статистичні тести з переліку.

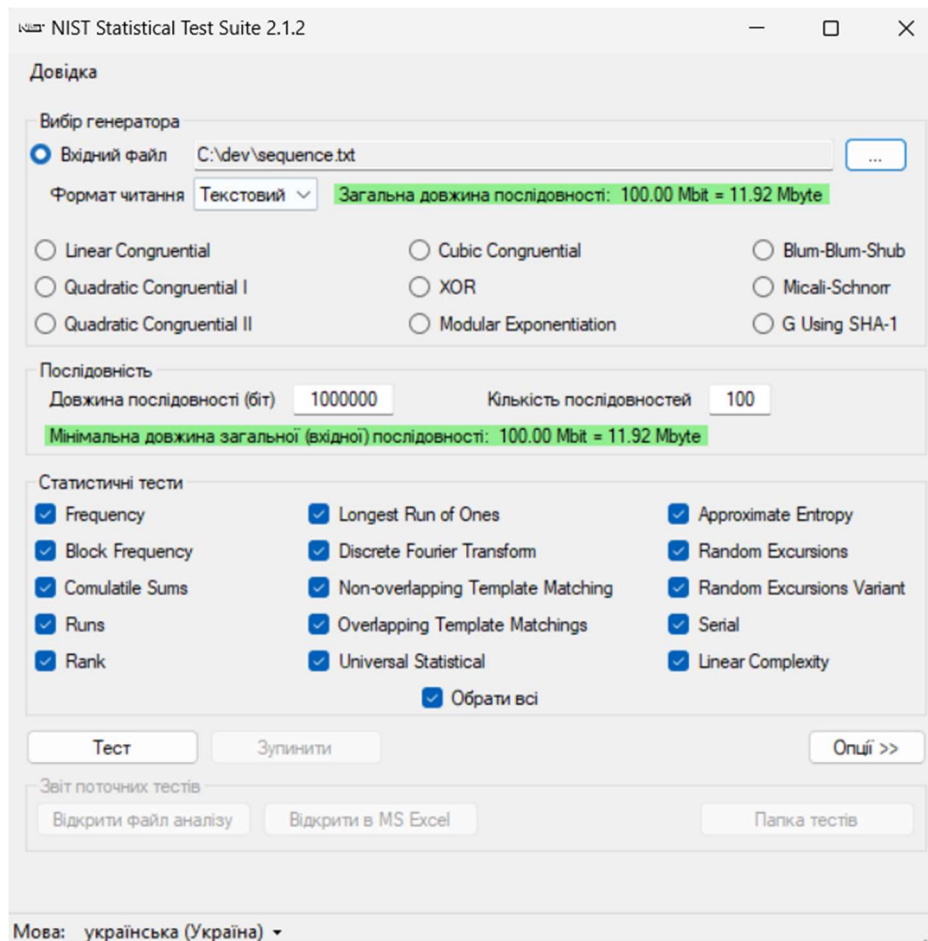


Рисунок 1.2 – Головне вікно програми NIST Statistical Test Suite 2.1.2 після успішного вибору вхідного файлу та формату читання

Щоб розпочати тестування, потрібно натиснути кнопку «Тест». По закінченню програма покаже повідомлення про завершення аналізу (рис.1.3). На формі з'явиться ProgressBar, який показує прогрес процесу тестування. Орієнтовний час виконання залежить від потужності комп'ютера та розміру вхідного файлу. Після завершення тестування система видасть повідомлення (рис. 1.3).

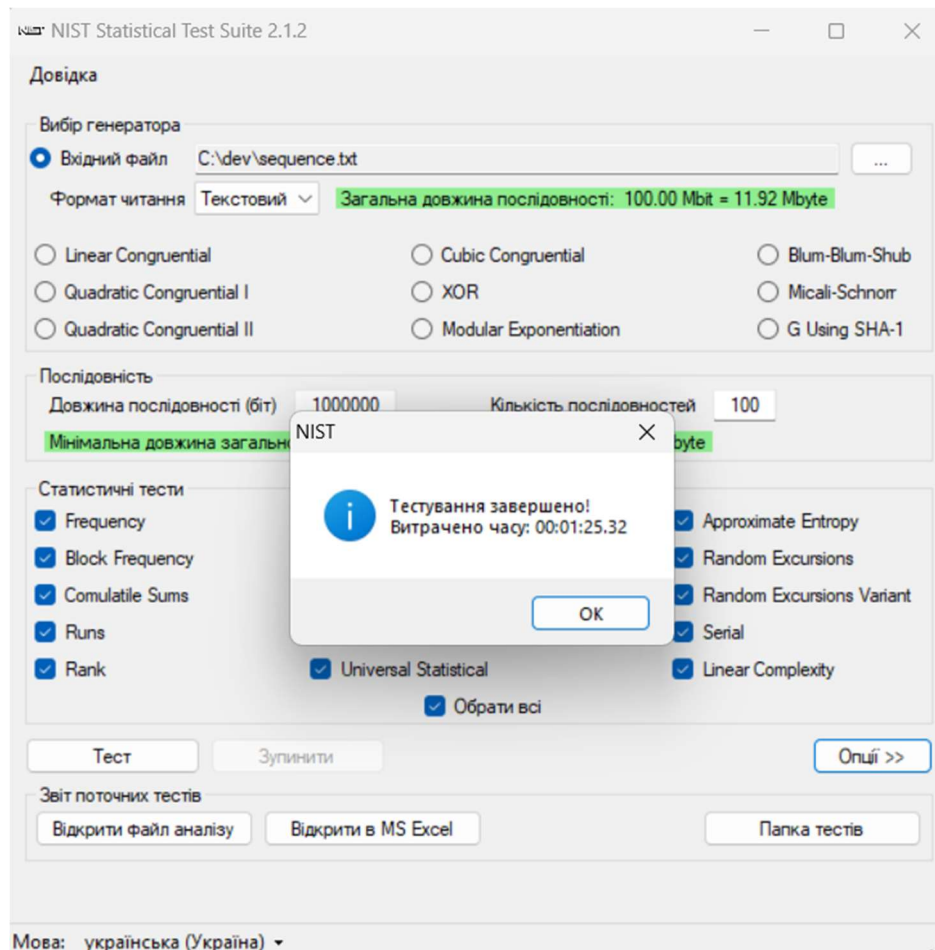


Рисунок 1.3 – Головне вікно програми NIST Statistical Test Suite 2.1.2 після успішного проведення тестувань

Перегляд результатів можливо провести у текстовому форматі та у вигляді таблиці MS Excel. Для перегляду текстового варіанту потрібно натиснути кнопку «Відкрити файл аналізу», а для перегляду звіту у MS Excel – «Відкрити в MS Excel».

Таким чином NIST Statistical Test Suite дозволяє автоматизувати перевірки бінарних послідовностей на відповідність вимогам випадковості, що є необхідним етапом аналізу генераторів псевдовипадкових чисел у криптографічних системах.

Висновки до розділу 1

У цьому розділі розглянуто концептуальний зв'язок між властивостями хаотичних систем і вимогами криптографії, що обґрунтовує доцільність використання детермінованого хаосу для розробки генераторів псевдовипадкових послідовностей. Далі проведено порівняльний аналіз динамічних систем за показниками інтенсивності хаосу та обчислювальною складністю: з відкритих джерел зібрано значення параметрів і показників, сформовано ієрархію за інтенсивністю хаосу та таблицю обчислювальної складності, а також зроблено висновки щодо їхньої придатності для використання у криптографічних системах. Проведено класифікацію використаних у роботі систем як змішаних динамічних детермінованих, оскільки вони базуються на поєднанні дискретного логістичного відображення з неперервними системами. Також виконано огляд традиційних генераторів бінарних послідовностей — генератора Блума–Мікалі та RSA-генератора — і проаналізовано їхні переваги та недоліки з погляду практичного застосування. Окремо розглянуто інструментарій NIST Statistical Test Suite, який є необхідним етапом аналізу генераторів псевдовипадкових чисел у криптографічних системах завдяки автоматизованій перевірці бінарних послідовностей на відповідність вимогам випадковості.

РОЗДІЛ 2. Моделювання та алгоритмізація гібридних систем шифрування

2.1 Математичне представлення та аналітичний опис хаотичних процесів

Для проведення розробки алгоритмів генераторів псевдовипадкових послідовностей у даному розділі зроблено математичний опис досліджуваних динамічних нелінійних систем в стані хаосу.

2.1.1 Одномірне логістичне відображення

$$x_{n+1} = rx_n(1 - x_n), \quad r = 4$$
$$x_{n+1} = 4x_n(1 - x_n)$$

2.1.2 Система Лоренца

Хаотична динаміка Лоренца. Описується системою диференціальних рівнянь першого порядку алгебраїчному вигляді:

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

де $\sigma = 10, \rho = 28, \beta = 8/3$. В алгебраїчному вигляді, зручному для алгоритмізації, система рівнянь має наступний вигляд:

$$\begin{cases} \dot{x} = 10(y - x) \\ \dot{y} = x(28 - z) - y \\ \dot{z} = xy - \frac{8}{3}z \end{cases}$$

Для розробки алгоритму на основі чисельного інтегрування використано алгоритм Рунге-Кутта (RK4).

20

$$k_{x1} = 10(y_n - x_n)$$

$$k_{y1} = x_n(28 - z_n) - y_n$$

$$k_{z1} = x_n y_n - \frac{8}{3} z_n$$

$$k_{x2} = 10\left((y_n + \frac{h}{2} k_{y1}) - (x_n + \frac{h}{2} k_{x1})\right)$$

$$k_{y2} = \left(x_n + \frac{h}{2} k_{x1}\right) \left(28 - \left(z_n + \frac{h}{2} k_{z1}\right)\right) - \left(y_n + \frac{h}{2} k_{y1}\right)$$

$$k_{z2} = (x_n + \frac{h}{2} k_{x1})(y_n + \frac{h}{2} k_{y1}) - \frac{8}{3} (z_n + \frac{h}{2} k_{z1})$$

$$k_{x3} = 10\left((y_n + \frac{h}{2} k_{y2}) - (x_n + \frac{h}{2} k_{x2})\right)$$

$$k_{y3} = \left(x_n + \frac{h}{2} k_{x2}\right) \left(28 - \left(z_n + \frac{h}{2} k_{z2}\right)\right) - \left(y_n + \frac{h}{2} k_{y2}\right)$$

$$k_{z3} = (x_n + \frac{h}{2} k_{x2})(y_n + \frac{h}{2} k_{y2}) - \frac{8}{3} (z_n + \frac{h}{2} k_{z2})$$

10

$$k_{x4} = 10((y_n + h k_{y3}) - (x_n + h k_{x3}))$$

$$k_{y4} = (x_n + h k_{x3})(28 - (z_n + h k_{z3})) - (y_n + h k_{y3})$$

$$k_{z4} = (x_n + h k_{x3})(y_n + h k_{y3}) - \frac{8}{3} (z_n + h k_{z3})$$

1

$$x_{n+1} = x_n + \frac{h}{6} (k_{x1} + 2k_{x2} + 2k_{x3} + k_{x4})$$

$$y_{n+1} = y_n + \frac{h}{6} (k_{y1} + 2k_{y2} + 2k_{y3} + k_{y4})$$

$$z_{n+1} = z_n + \frac{h}{6} (k_{z1} + 2k_{z2} + 2k_{z3} + k_{z4})$$

2.1.3 Система Ресслера

3

$$\begin{cases} \frac{dx}{dt} = -y - z \\ \frac{dy}{dt} = x + ay \\ \frac{dz}{dt} = b + z(x - c) \end{cases}$$

$$a = 0,2; b = 0,2; c = 5,7$$

14

$$\begin{cases} \dot{x} = -y - z \\ \dot{y} = x + 0,2y \\ \dot{z} = 0,2 + z(x - 5,7) \end{cases}$$

$$k_{x1} = -y_n - z_n$$

$$k_{y1} = x_n + 0,2y_n$$

$$k_{z1} = 0,2 + z_n(x_n - 5,7)$$

$$k_{x2} = -\left(y_n + \frac{h}{2}k_{y1}\right) - \left(z_n + \frac{h}{2}k_{z1}\right)$$

$$k_{y2} = \left(x_n + \frac{h}{2}k_{x1}\right) + 0,2\left(y_n + \frac{h}{2}k_{y1}\right)$$

$$k_{z2} = 0,2 + \left(z_n + \frac{h}{2}k_{z1}\right)\left(\left(x_n + \frac{h}{2}k_{x1}\right) - 5,7\right)$$

13

$$k_{x3} = -\left(y_n + \frac{h}{2}k_{y2}\right) - \left(z_n + \frac{h}{2}k_{z2}\right)$$

$$k_{y3} = \left(x_n + \frac{h}{2}k_{x2}\right) + 0,2\left(y_n + \frac{h}{2}k_{y2}\right)$$

$$k_{z3} = 0,2 + \left(z_n + \frac{h}{2}k_{z2}\right)\left(\left(x_n + \frac{h}{2}k_{x2}\right) - 5,7\right)$$

$$k_{x4} = -\left(y_n + hk_{y3}\right) - \left(z_n + hk_{z3}\right)$$

$$k_{y4} = \left(x_n + hk_{x3}\right) + 0,2\left(y_n + hk_{y3}\right)$$

$$k_{z4} = 0,2 + \left(z_n + hk_{z3}\right)\left(\left(x_n + hk_{x3}\right) - 5,7\right)$$

1

$$x_{n+1} = x_n + \frac{h}{6}(k_{x1} + 2k_{x2} + 2k_{x3} + k_{x4})$$

$$y_{n+1} = y_n + \frac{h}{6}(k_{y1} + 2k_{y2} + 2k_{y3} + k_{y4})$$

$$z_{n+1} = z_n + \frac{h}{6}(k_{z1} + 2k_{z2} + 2k_{z3} + k_{z4})$$

2.1.4 Система Чua

$$f(x) = m_1 x + \frac{1}{2}(m_0 - m_1)(|x + 1| - |x - 1|)$$

$$\begin{cases} \frac{dx}{dt} = \alpha(y - x - f(x)) \\ \frac{dy}{dt} = x - y + z \\ \frac{dz}{dt} = -\beta y \end{cases}$$

$$\alpha = 15,6; \beta = 28; m_0 = -\frac{8}{7}; m_1 = -\frac{5}{7}$$

$$\begin{cases} \dot{x} = 15,6(y - x - f(x)) \\ \dot{y} = x - y + z \\ \dot{z} = -28y \end{cases}$$

$$k_{x1} = 15,6(y_n - x_n - f(x_n))$$

$$k_{y1} = x_n - y_n + z_n$$

$$k_{z1} = -28y_n$$

$$k_{x2} = 15,6((y_n + \frac{h}{2}k_{y1}) - (x_n + \frac{h}{2}k_{x1}) - f(x_n + \frac{h}{2}k_{x1}))$$

$$k_{y2} = (x_n + \frac{h}{2}k_{x1}) - (y_n + \frac{h}{2}k_{y1}) + (z_n + \frac{h}{2}k_{z1})$$

$$k_{z2} = -28(y_n + \frac{h}{2}k_{y1})$$

$$k_{x3} = 15,6((y_n + \frac{h}{2}k_{y2}) - (x_n + \frac{h}{2}k_{x2}) - f(x_n + \frac{h}{2}k_{x2}))$$

$$k_{y3} = (x_n + \frac{h}{2}k_{x2}) - (y_n + \frac{h}{2}k_{y2}) + (z_n + \frac{h}{2}k_{z2})$$

$$k_{z3} = -28(y_n + \frac{h}{2}k_{y2})$$

$$k_{x4} = 15,6((y_n + hk_{y3}) - (x_n + hk_{x3}) - f(x_n + hk_{x3}))$$

$$k_{y4} = (x_n + hk_{x3}) - (y_n + hk_{y3}) + (z_n + hk_{z3})$$

$$k_{z4} = -28(y_n + hk_{y3})$$

$$x_{n+1} = x_n + \frac{h}{6}(k_{x1} + 2k_{x2} + 2k_{x3} + k_{x4})$$

$$y_{n+1} = y_n + \frac{h}{6}(k_{y1} + 2k_{y2} + 2k_{y3} + k_{y4})$$

$$z_{n+1} = z_n + \frac{h}{6}(k_{z1} + 2k_{z2} + 2k_{z3} + k_{z4})$$

2.1.5 Осцилятор Дуффінга

$$\frac{d^2x}{dt^2} + \delta \frac{dx}{dt} + \alpha x + \beta x^3 = \gamma \cos(\omega t)$$

$$\alpha = 1; \beta = -1; \delta = 0,2; \gamma = 0,3; \omega = 1$$

$$\begin{cases} \dot{x} = y \\ \dot{y} = -0,2y - x + x^3 + 0,3 \cos(\omega t) \end{cases}$$

$$k_{x1} = y_n$$

$$k_{y1} = -0,2y_n - x_n + x_n^3 + 0,3 \cos(t_n)$$

$$k_{x2} = y_n + \frac{h}{2}k_{y1}$$

$$k_{y2} = -0,2(y_n + \frac{h}{2}k_{y1}) - (x_n + \frac{h}{2}k_{x1}) + (x_n + \frac{h}{2}k_{x1})^3 + 0,3 \cos(t_n + \frac{h}{2})$$

$$k_{x3} = y_n + \frac{h}{2}k_{y2}$$

$$k_{y3} = -0,2(y_n + \frac{h}{2}k_{y2}) - (x_n + \frac{h}{2}k_{x2}) + (x_n + \frac{h}{2}k_{x2})^3 + 0,3 \cos(t_n + \frac{h}{2})$$

$$k_{x4} = y_n + hk_{y3}$$

$$k_{y4} = -0,2(y_n + hk_{y3}) - (x_n + hk_{x3}) + (x_n + hk_{x3})^3 + 0,3 \cos(t_n + h)$$

$$x_{n+1} = x_n + \frac{h}{6}(k_{x1} + 2k_{x2} + 2k_{x3} + k_{x4})$$

$$y_{n+1} = y_n + \frac{h}{6}(k_{y1} + 2k_{y2} + 2k_{y3} + k_{y4})$$

$$t_{n+1} = t_n + h$$

2.1.6 Осцилятор Ван дер Поля

$$\begin{cases} \frac{dx}{dt} = y \\ \frac{dy}{dt} = \mu(1 - x^2) \frac{dx}{dt} - \omega_0^2 x + 2\pi \cos(0,7t) \end{cases}$$

$$\mu = 3,4, \omega_0^2 = \pi, A = 2\pi, \omega = 0,7$$

$$\begin{cases} \dot{x} = y \\ \dot{y} = 3,4(1 - x^2)y - \pi x + 2\pi \cos(0,7t) \end{cases}$$

$$k_{x1} = y_n$$

$$k_{y1} = 3,4(1 - x_n^2)y_n - \pi x_n + 2\pi \cos(0,7t_n)$$

$$k_{x2} = y_n + \frac{h}{2}k_{y1}$$

$$k_{y2} = 3,4(1 - (x_n + \frac{h}{2}k_{x1})^2)(y_n + \frac{h}{2}k_{y1}) - \pi(x_n + \frac{h}{2}k_{x1}) + 2\pi \cos(0,7(t_n + \frac{h}{2}))$$

$$k_{x3} = y_n + \frac{h}{2}k_{y2}$$

$$k_{y3} = 3,4(1 - (x_n + \frac{h}{2}k_{x2})^2)(y_n + \frac{h}{2}k_{y2}) - \pi(x_n + \frac{h}{2}k_{x2}) + 2\pi \cos(0,7(t_n + \frac{h}{2}))$$

$$k_{x4} = y_n + hk_{y3}$$

$$k_{y4} = 3,4(1 - (x_n + hk_{x3})^2)(y_n + hk_{y3}) - \pi(x_n + hk_{x3}) + 2\pi \cos(0,7(t_n + h))$$

$$x_{n+1} = x_n + \frac{h}{6}(k_{x1} + 2k_{x2} + 2k_{x3} + k_{x4})$$

$$y_{n+1} = y_n + \frac{h}{6}(k_{y1} + 2k_{y2} + 2k_{y3} + k_{y4})$$

$$t_{n+1} = t_n + h$$

2.1.7 Маятник з силою, що змушує (Forced Pendulum)

$$\begin{cases} \frac{d\theta}{dt} = v \\ \frac{dv}{dt} = -q \frac{d\theta}{dt} - \sin(\theta) + F \cos(\omega t) \end{cases}$$

$$\begin{cases} \dot{\theta} = v \\ \dot{v} = -0,5v - \sin(\theta) + 1,2 \cos\left(\frac{2}{3}t\right) \end{cases}$$

$$q = 0,5; \omega = \frac{2}{3}; F = 1,2$$

$$k_{\theta 1} = v_n$$

$$k_{v1} = -0,5v_n - \sin(\theta_n) + 1,2 \cos\left(\frac{2}{3}t_n\right)$$

$$k_{\theta 2} = v_n + \frac{h}{2}k_{v1}$$

$$k_{v2} = -0,5\left(v_n + \frac{h}{2}k_{v1}\right) - \sin\left(\theta_n + \frac{h}{2}k_{\theta 1}\right) + 1,2 \cos\left(\frac{2}{3}\left(t_n + \frac{h}{2}\right)\right)$$

$$k_{\theta 3} = v_n + \frac{h}{2}k_{v2}$$

$$k_{v3} = -0,5\left(v_n + \frac{h}{2}k_{v2}\right) - \sin\left(\theta_n + \frac{h}{2}k_{\theta 2}\right) + 1,2 \cos\left(\frac{2}{3}\left(t_n + \frac{h}{2}\right)\right)$$

$$k_{\theta 4} = v_n + hk_{v3}$$

$$k_{v4} = -0,5(v_n + hk_{v3}) - \sin(\theta_n + hk_{\theta 3}) + 1,2 \cos\left(\frac{2}{3}(t_n + h)\right)$$

$$\theta_{n+1} = \theta_n + \frac{h}{6}(k_{\theta 1} + 2k_{\theta 2} + 2k_{\theta 3} + k_{\theta 4})$$

$$v_{n+1} = v_n + \frac{h}{6}(k_{v1} + 2k_{v2} + 2k_{v3} + k_{v4})$$

$$t_{n+1} = t_n + h$$

2.2 Алгоритмічний опис систем шифрування на основі динамічного хаосу

2.2.1 Алгоритмічна реалізація генераторів псевдовипадкових послідовностей на основі хаотичної динаміки

Для забезпечення шифраторів даними для шифрування файлів розроблено генератори псевдовипадкових послідовностей наступним чином. Кожен генератор реалізовано шляхом використання комбінації Логістичного відображення як дискретної системи у стані хаосу та відповідної неперервної хаотичної системи. Така комбінація являє собою гібридну систему, що працює за принципом впливу дискретної системи на неперервну.

Генерація псевдовипадкових послідовностей на основі гібридних систем здійснюється наступним чином:

- користувачем вводяться початкові умови для чотирьох змінних (для логістичного відображення від 0 до 1, три довільні для обраної неперервної системи);
- на основі початкових умов програма робить розрахунок нових значень для логістичного відображення та обраної неперервної системи хаосу (RK4);
- на основі нового результату логістичної змінної здійснюється модифікація неперервної системи шляхом вибіркового арифметичного поєднання логістичної змінної з обраною змінною неперервної системи;
- складаються всі значення цього кроку циклу;
- результат додавання в залежності від необхідності використовується для генерації псевдовипадкового числа або байтової послідовності на основі SHA256;
- далі цикл повторюється.

Алгоритм роботи генератора докладно описано блок-схемою на рисунку 2.1. Робота кожного генератора починається з введення довжини послідовності, яка генерується, початкових станів для систем хаосу, а також вибору байтового або числового режиму генерування через змінну isSha256.

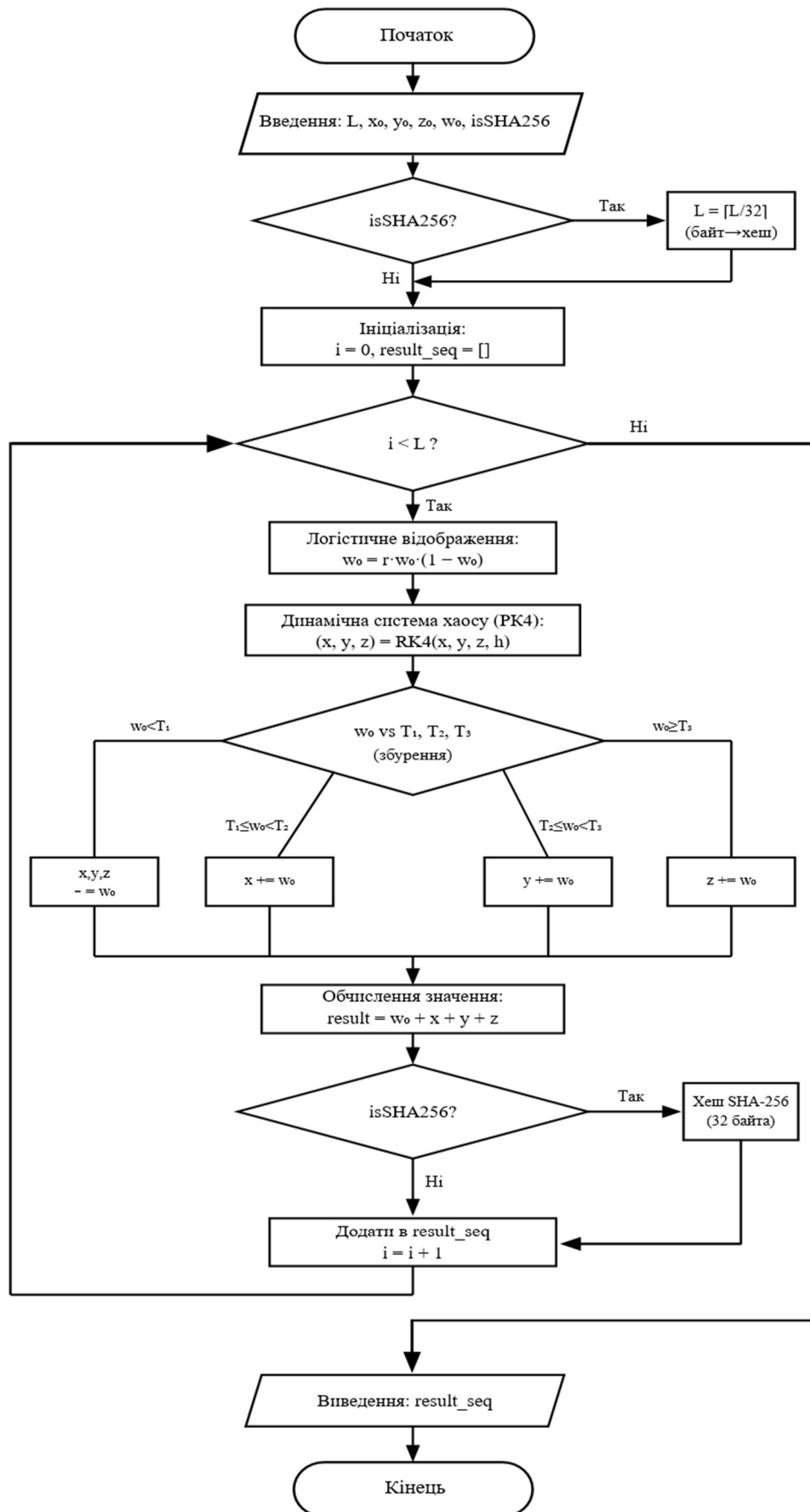


Рисунок 2.1 – Блок-схема генератора псевдохаотичної послідовності

Наступним кроком залежно від режиму генерації здійснюється модифікація значення довжини генерованої послідовності. Якщо встановлено байтовий режим, тоді алгоритм виконує ділення довжини послідовності на 32. За іншого режиму (числового) значення довжини не змінюється.

На наступному етапі відбувається генерація кожного елементу випадкової послідовності всередині циклу. Спочатку формується наступне значення w_0 та наступний стан x , y , z неперервної хаотичної системи шляхом дискретизації за допомогою методу Рунге-Кутта (RK4). На основі w_0 здійснюються модифікації стану неперервної хаотичної системи. Для цього використовуються межі інтервалів T_1 , T_2 , T_3 для формування 4 рівновірогідних областей в межах інтервалу $(0;1)$ функції розподілу системи Логістичного відображення [32]. На основі визначених меж проводиться збурення шляхом додавання або віднімання w_0 від змінних стану неперервної системи відповідно до умов. Якщо w_0 менше за T_1 , віднімаємо w_0 від всіх змінних, якщо між T_1 і T_2 додаємо до змінної x , якщо між T_2 і T_3 – до змінної y , якщо більше за T_3 – до змінної z . Наступним кроком обчислюємо суму стану w_0 та x , y , z .

Далі за умови вибору посимвольного режиму, результат суми додається до результуючої послідовності. Якщо ж обрано байтовий формат, дані проходять процес хешування за допомогою алгоритму SHA-256 та додаються в результуючий масив. Далі цикл продовжується допоки не буде згенеровано всю послідовність. Після виконання циклу відбувається виведення результату і завершується процес виконання алгоритму.

Детермінований хаос накладає на умови використання генератора псевдовипадкових послідовностей певні обмеження у вигляді чітко визначеного діапазону початкових умов. Це пов'язано з такими особливостями хаотичних систем, як:

– вікна періодичності та мультистабільність. Всередині хаотичних областей параметрів можуть існувати вузькі діапазони початкових умов, за яких траєкторія системи може мати точки рівноваги, граничні цикли, інваріантні

множини і поверхні, що розділяють (наприклад, стабільні різноманіття сідел), інші дивні атрактори;

– вихід початкових координат за межі басейну притягання дивного атрактора призводить до необмеженого зростання значень (уходу в нескінченність);

– чутливість до кроку інтегрування. Використання методу Рунге-Кутта 4-го порядку (RK4) вимагає фіксації кроку h . Перевищення критичного значення h порушує детермінізм та стійкість атрактора.

З урахуванням даних обмежень, за допомогою ШІ підібрано діапазони початкових умов для хаотичних систем, які забезпечують стабільний стан детермінованого хаосу генерації псевдопослідовностей. Дані зведені до таблиці 2.1.

Таблиця 2.1 – Зведена таблиця безпечних діапазонів початкових умов

Система	Початкові умови	Параметри	Безпечний діапазон
Лоренц	x, y, z	$\sigma=10; \rho=28; \beta=8/3$	$x \in [-20;20];$ $y \in [-20;20]; z \in [0;60]$
Ресслер	x, y, z	$a=0,2; b=0,2; c=5,7$	$x \in [-15;15];$ $y \in [-15;15]; z \in [0;25]$
Чуа	x, y, z	$\alpha=15,6; \beta=28$	$x \in [-3;3]; y \in [-2;2];$ $z \in [-3;3]$
Дуффінг	x, y	$\delta=0,2; \gamma=0,3; \omega=1$	$x \in [-2;2]; y \in [-2;2]$
Ван-дер-Поль	x, y	$\mu=3,4; \omega_0^2=\pi$	$x \in [-3;3]; y \in [-8;8]$
Forced Pendulum	θ, v	$q=0,5; F=1,2; \omega=2/3$	$\theta \in [-\pi;\pi]; v \in [-6;6]$

Дані діапазони використанні у інтерфейсі користувача для коректного введення даних.

2.2.2 Алгоритм шифрування та розшифрування зображень

Шифрування зображення виконано за допомогою алгоритму фрактальних сортувальних матриць [33], який використовує два криптографічні методи:

скремблювання та дифузію. Під час скремблювання відбувається обмін позиціями пікселів у просторовій області згідно з псевдовипадковою послідовністю. Ця операція допомагає зруйнувати просторову кореляцію між пікселями у вихідному файлі, що є першим етапом криптографічного захисту зображення. А у процесі дифузії відбувається зміна значення кожного пікселя (також за правилом детермінованого хаосу) з метою забезпечення максимальної надійності шифрування інформації.

Алгоритм шифрування працює у наступний спосіб:

- зображення зчитується в матрицю пікселів P ;
 - матриця P доповнюється до квадратної шляхом копіювання частини зображення;
 - на основі розміру зображення та певних початкових умов генерується псевдовипадкова числова послідовність відповідної довжини, яка поділяється на послідовності C_1 , C_2 та C_3 ;
 - на наступному кроці вибирається початкова матриця A_1 , яка є основою для ітеративної генерації фрактальної сортувальної матриці FSM необхідних розмірів (A_n);
 - з послідовності C_1 формується за допомогою функції $\text{sort}(C_1)$ проміжна матриця S розміру $N \times N$.
 - далі відбувається скремблінг (перемішування) матриці P за допомогою матриць A_n і S ;
 - для забезпечення дифузії формується масив з $T(i)$ на основі послідовності C_2 та S_3 шляхом сортування C_3 за допомогою функції $\text{sort}(C_3)$;
 - в циклі відбувається вибір кожного елементу скрембльованої матриці за індексом $S_3(i)$ та операція XOR з відповідним елементом $T(i)$ на першій ітерації.
 - на наступних ітераціях виконується побайтовий XOR між $T(i)$, $S(i)$ та значенням пікселя результуючої матриці, отриманим на попередньому кроці.
- Етапи дифузії здійснюються за допомогою формул 2.1:

$$P_1(i) = \begin{cases} T(i) \oplus P_0(S_3(i)), i = 1 \\ P_1(i-1) \oplus T(i) \oplus P_0(S_3(i)), i \geq 2 \end{cases} \quad (2.1)$$

$$T(i) = \text{mod}([C_2(i) \times 10^5], 256)$$

$$S_3 = \text{sort}(C_3)$$

– цикл продовжується до отримання кінцевої матриці.

Алгоритм розшифрування реалізовано за тим самим принципом, що й шифрування, але у зворотному порядку. Оскільки ключі для шифрування та розшифрування збігаються, а FSM використовується узгоджено на обох етапах, відтворюється ідентична послідовність керувальних значень. Крім того, операції скремблювання та дифузії є оборотними, тобто для кожного кроку шифрування існує відповідний зворотний крок. Отже, застосування зворотної послідовності операцій забезпечує повне відновлення початкового зображення, а розшифрування є оберненим процесом до алгоритму шифрування.

Оскільки скремблювання виконується на основі FSM, далі наведено алгоритм формування цієї матриці. На першому кроці алгоритму отримується матриця другого порядку, що складається з елементів 1,2,3,4. Далі формується наступна матриця 4-го порядку шляхом ітерування над A_1 , за формулою:

$$A_0^2 = \begin{bmatrix} A_{1,1}^2 & A_{1,2}^2 \\ A_{2,1}^2 & A_{2,2}^2 \end{bmatrix} \quad (2.2)$$

де $A_{i,j}^2 = (2^2 + 1)^{-1} \times A^1 + A^1(i,j)$ для $(i = 1, 2; j = 1, 2)$, $A_{i,j}$ є підблоком в i -тому рядку та j -му стовбці матриці A , а $A(i,j)$ – елемент в i -ому рядку та j -му стовбці матриці A .

На наступному кроці отримуємо FSM шляхом сортування A_0^2 від меншого до більшого:

$$A^2 = \text{order}(A_0^2) \quad (2.3)$$

де $\text{order}(\cdot)$ – це функція сортування матриці від меншого до більшого у відповідності до значення елементів у матриці.

Далі формуємо матрицю $A_0^n \in R^{2^n \times 2^n}$ m_{n-1} порядку, що отримана шляхом ітерації над $A^{n-1} \in R^{2^{n-1} \times 2^{n-1}}$ за допомогою наступної формули:

$$A_0^n = \begin{bmatrix} A_{1,1}^n & A_{1,2}^n & \cdots & A_{1,m_{n-1}}^n \\ A_{2,1}^n & A_{2,2}^n & \cdots & A_{2,m_{n-1}}^n \\ \vdots & \ddots & \ddots & \vdots \\ A_{m_{n-1},1}^n & A_{m_{n-1},2}^n & \cdots & A_{m_{n-1},m_{n-1}}^n \end{bmatrix} \quad (2.4)$$

де кожен елемент розраховується за $A_{i,j}^n = (m_n^2 + 1)^{-1} \times A^1 + A^{n-1}(i, j)$ для $(i = 1, \dots, m_{n-1}; j = 1, \dots, m_{n-1})$

Для кожної ітерації отримуємо A^n наступним чином:

$$A^n = \text{order}(A_0^n) \quad (2.5)$$

Таким чином, формується сортувальна матриця за фрактальним принципом (детермінована хаотичність та самовідтворюваність).

Алгоритм формування FSM було модифіковано шляхом заміни множника з дробового $(m_n^2 + 1)^{-1}$ на цілочисельні $(m_n^2 + 1)$. Дані зміни було введено з метою оптимізації алгоритму.

Програмна реалізація алгоритму шифрування докладно описана блок-схемою на рисунку 2.2.

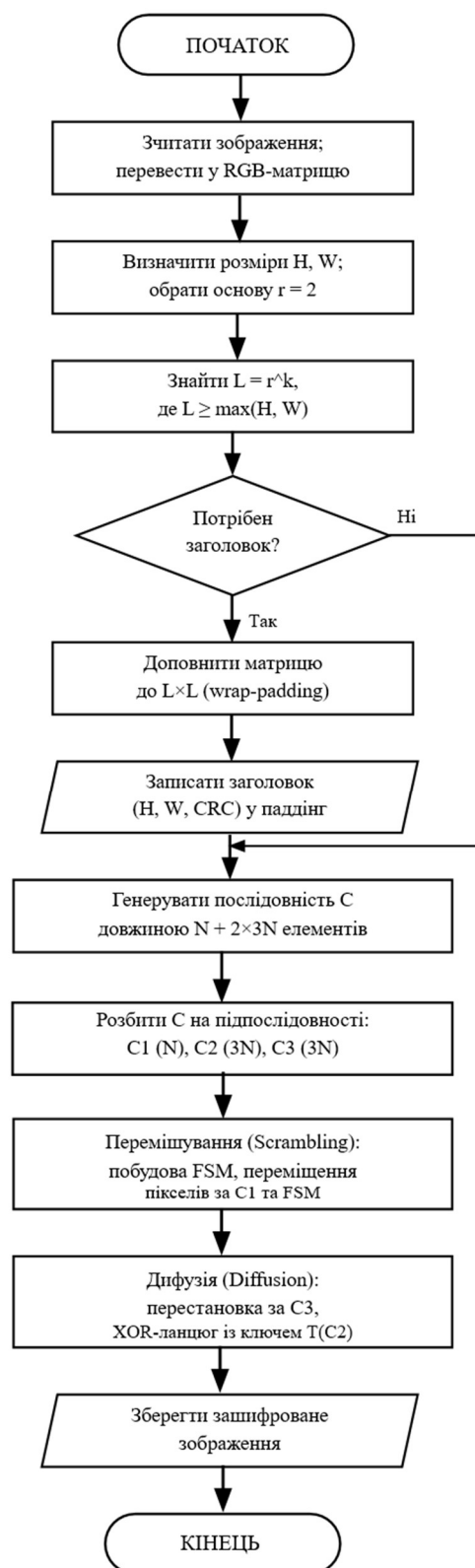


Рисунок 2.2 – Блок-схема алгоритму шифрування зображення

Алгоритм починає свою роботу із зчитування зображення у матрицю пікселів. Наступним кроком є визначення розмірів початкової та вибір

початкового розміру сторони для матриці FSM. Початковим розміром сторони є $r=2$, оскільки для реалізації навчального проєкту за основу було взято матрицю 2×2 .

Наступним кроком знаходиться величина сторони кінцевої матриці FSM. Далі перевіряється потреба у заголовку, в якому лежатиме інформація про початковий розмір матриці, для виконання розшифрування. За потреби заголовку матриця доповнюється додатковими пікселями за розміром L .

Далі створюється заголовок та записується замість частини доповнених пікселів. На наступному кроці генерується загальна послідовність, що розбивається на послідовності C_1 , C_2 та C_3 . На наступних кроках масив зображення проходить через процедури скремблінгу та дифузії. Після виконання дифузії відбувається збереження отриманого зображення.

Розшифрування здійснюється за тим самим принципом, але в оберненому порядку, що забезпечує відновлення початкового зображення.

2.2.3 Алгоритми шифрування та розшифрування тексту

Шифрування тексту в системі реалізовано у двох режимах: побайтовому та посимвольному. Вибір здійснюється користувачем під час введення даних на стороні клієнта. Відповідно до обраного режиму викликається відповідна підфункція шифрування.

Процес побайтового шифрування починається з перетворення тексту у байтовий формат. Далі генерується байтова послідовність, довжина якої дорівнює довжині тексту у байтах. Після цього елементи обох послідовностей, поєднуються в нову шляхом виконання побайтової операції XOR. У результаті формується зашифрована байтова послідовність, яка виводиться у вигляді шифротексту. Посимвольне шифрування розпочинається з генерації послідовності, розміром $2n$, де n довжина тексту в символах.

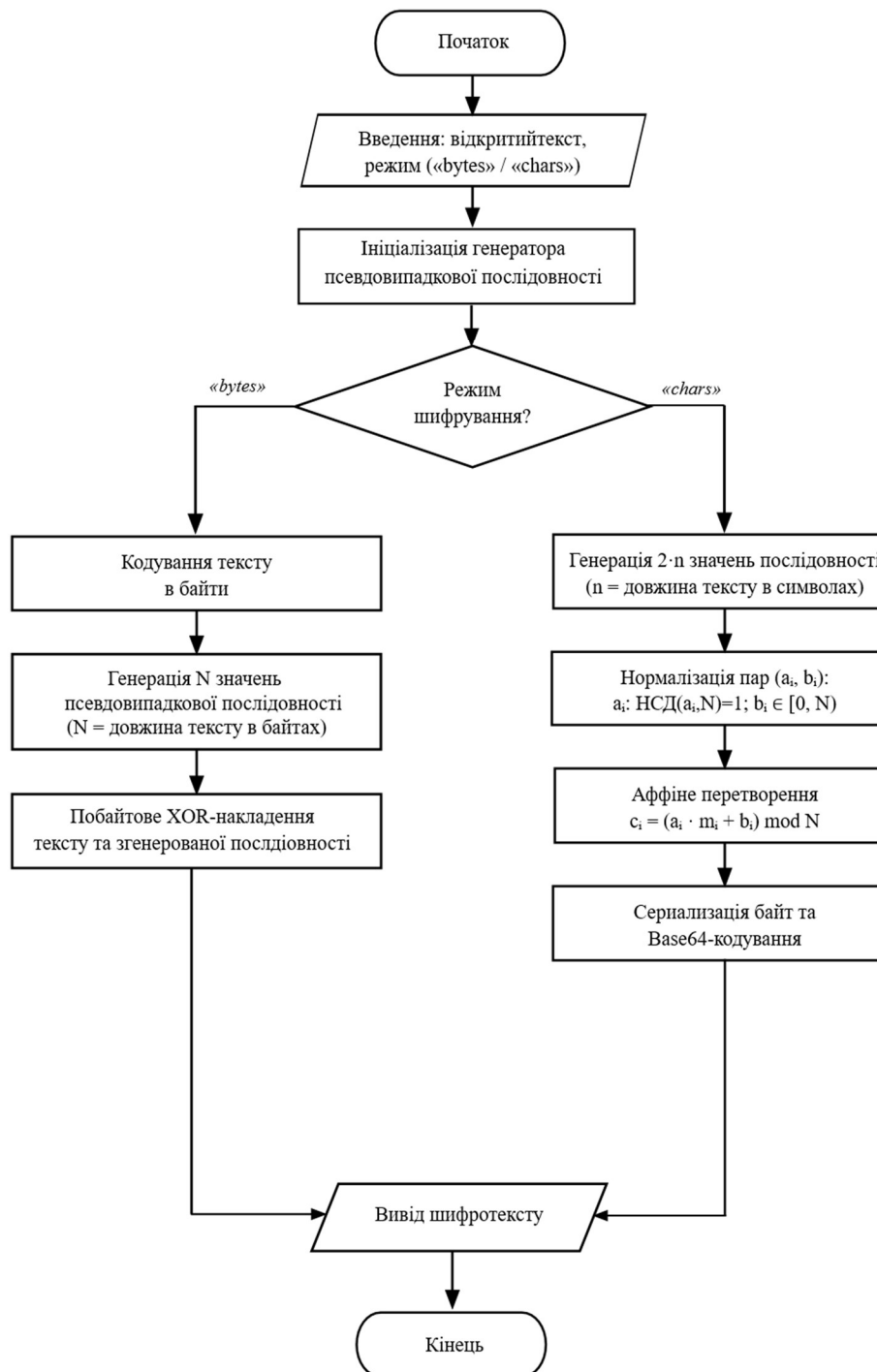


Рисунок 2.3 – Блок-схема алгоритму шифрування тексту

На наступному етапі відбувається нормалізація елементів масиву, оскільки кожна неперекривна пара задає коефіцієнти a та b для афінного перетворення, де a є першим елементом пари, b — другим. Коефіцієнт a приводиться до значення, яке є взаємно простим з N , де N — максимально можливе значення функції

афінного перетворення. дорівнює одиниці. Для коефіцієнту b виконується приведення до діапазону від 0 до N . Після цього відбувається посимвольне афінне перетворення: код кожного символу множиться на a , до отриманого результату додається b , а далі проводиться обчислення остачі за модулем N .

На завершальному етапі послідовність перетворюється у байтовий формат та кодується за допомогою base64. Це дає змогу демонструвати результати шифрування, без проблем з виводом службових та непечатних символів.

Програмна реалізація алгоритму шифрування докладно описана блок-схемою на рисунку 2.3.

Алгоритм побайтового розшифрування реалізується аналогічно алгоритму побайтового шифрування, оскільки операція XOR є оборотною: повторне застосування тієї самої ключової послідовності до шифротексту дозволяє відновити вихідні дані.

У свою чергу, посимвольний розшифровувач працює за оберненим до шифрування принципом. Спочатку зашифровані дані перетворюються назад у коди символів. Далі кожен код зашифрованого символу проходить процедуру, обернену до афінного перетворення: від значення віднімається b , а результат множиться на число, обернене до a за модулем N . Після цього отримані значення подаються у символьному форматі.

2.2.4 Алгоритми шифрування та розшифрування аудіофайлів

Даний алгоритм шифрування реалізовано виключно для файлів формату WAV, оскільки цей формат є нестисненим, а дані в ньому зберігаються у вигляді значень амплітуди та частоти дискретизації (тобто кількості фіксувань сигналу за секунду). Формати MP3 та MP4 використовують алгоритми. стиснення, що можуть обрізати зашифровані дані, сприймаючи їх як шум. Тому такі файли шифруються як звичайні файли за алгоритмом, описаним у пункті 2.5.



Рисунок 2.4 – Блок-схема алгоритму шифрування аудіо

Алгоритм шифрування аудіофайлу працює наступним чином. Спочатку сервер отримує вхідний аудіофайл від клієнта. Далі виконується зчитування даних аудіофайлу в масив цілих чисел. На наступному етапі ці дані перетворюються у байтовий масив для приведення до формату, що

використовується в процесі шифрування. На основі довжини отриманого масиву формується псевдовипадкова послідовність байтів. Після цього виконується шифрування шляхом побайтового застосування операції XOR між байтовим масивом і згенерованою послідовністю. На завершальному етапі зашифровані дані перетворюються назад в аудіофайл, що дає змогу представити результат у зручній для демонстрації формі.

Алгоритм розшифрування аудіоданих реалізовано аналогічно алгоритму шифрування, оскільки операція XOR є оборотною, тобто застосування тієї самої ключової послідовності до шифротексту дозволяє відновити вихідні дані.

Програмна реалізація алгоритму шифрування докладно описана блок-схемою на рисунку 2.4.

2.2.5 Алгоритми шифрування та розшифрування файлів

Алгоритм шифрування файлів працює наступним чином. Спочатку серверна частина програми отримує вхідний файл від клієнта та перетворює його на послідовність байт для подальшого шифрування. На основі довжини отриманого масиву формується псевдовипадкова послідовність байтів, яка об'єднується у байтовий рядок. Після цього виконується шифрування шляхом побайтового застосування операції XOR між байтовою та згенерованою послідовностями. На завершальному етапі зашифровані дані перетворюються назад в у файл, що дає змогу представити результат у зручній для демонстрації формі.

Програмна реалізація алгоритму шифрування докладно описана блок-схемою на рисунку 2.5 .

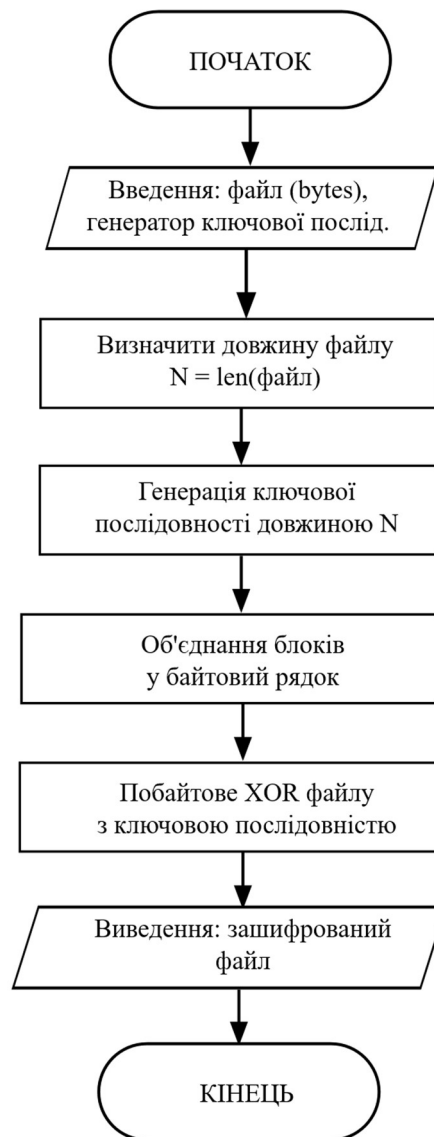


Рисунок 2.5 – Блок-схема алгоритму шифрування файлів

Алгоритм розшифрування файлів реалізовано аналогічно алгоритму шифрування, оскільки операція XOR є оборотною, тобто застосування тієї самої ключової послідовності до шифротексту дозволяє відновити вихідні дані.

2.3 Тестування та зрівняльний аналіз криптографічних генераторів на основі хаотичних систем

2.3.1 Статистична оцінка розроблених криптографічних систем за допомогою NIST STS

Тестування із застосуванням пакета NIST STS для оцінки розроблених криптографічних систем реалізовано за допомогою Excel-файлів зі структурованими даними результатів дослідження та додаткових скриптів для формування бінарних послідовностей.

Генерація бітової послідовності для кожного генератора здійснювалась у двох режимах: формування бітової послідовності в комбінації з алгоритмом хешування SHA-256, а також пряме перетворення чисел у біти.

Для кожного режиму було сформовано по два файли різних форматів: дані згенеровані за різних початкових станів та з одним початковим станом. Таким чином, статистично проаналізовано наступні режими бітових послідовностей:

- режим без хешування з одним початковим станом (Float/one seed);
- режим без хешування за різних початкових станів (Float/multiple seeds);
- режим з хешуванням з одним початковим станом (SHA-256/one seed);
- режим з хешуванням за різних початкових станів (SHA-256/multiple seeds);

Формат із різними початковими станами сформовано шляхом зміни початкового стану для кожної послідовності обсягом 1 мегабіт, тобто загалом використано 55 різних станів. Такий формат тестування проводився для комплексної перевірки ступеня залежності випадковості від початкових параметрів для підтвердження того, що система залишається хаотичною не лише за окремих параметрів, а й у межах усього діапазону допустимих початкових станів, у яких система перебуває у стані хаосу.

Формат з одним початковим станом використовувався для проведення експериментів, спрямованих на перевірку здатності системи зберігати хаотичні властивості під час обробки даних за незмінного початкового стану параметрів.

Після генерації кожен отриманий результат збережено в окремий бінарний файл обсягом 55 Мбіт відповідно до стандартних рекомендацій щодо тестування генераторів за допомогою пакета NIST STS.

Основною метрикою системи тестування NIST STS є p -value. Це статистичний показник, який показує, наскільки результат тесту узгоджується з припущенням, що протестована послідовність є випадковою за критерієм відповідного тесту NIST STS. Якщо значення p -value є достатньо великим, послідовність вважається такою, що пройшла тест; якщо p -value менше за заданий рівень значущості, це свідчить про статистично значуще відхилення від випадкової поведінки.

Також як метрика в системі тестування NIST STS використовується $proportion$. Це статистичний показник, який характеризує частку протестованих послідовностей, що пройшли відповідний тест з пакету NIST STS. Іншими словами, $proportion$ показує, яка частина згенерованих послідовностей має результат p -value не менший за заданий рівень значущості. Якщо значення $proportion$ є достатньо великим, результати тестування вважаються прийнятними; якщо $proportion$ нижче встановленої допустимої межі, це свідчить про недостатню кількість послідовностей, які пройшли тест, і може вказувати на відхилення від випадкової поведінки генератора.

Для кожного бінарного файлу проведено тестування за допомогою пакета NIST STS у режимі 55 послідовностей по 1 мільйону біт. Результати тестування сформовано у вигляді текстових файлів у форматі, наведеному на рисунку 2.6.

Комірки C1–C10 відображають розподіл p -value за десятьма рівними інтервалами діапазону $[0;1]$. Такий розподіл використовується для перевірки рівномірності отриманих p -value у межах відповідного статистичного тесту за допомогою P-VALUE. P-VALUE є показником статистичної узгодженості розподілу отриманих p -value з рівномірним розподілом. А показник

PROPORTION, у свою чергу, відображає частку послідовностей, які успішно пройшли відповідний тест. STATISTICAL TEST – назва тесту.

RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES												
generator is <C:\Users\sasha\Desktop\Диплом 25\ChaosCipher\DiplomAnalys\Експеримент\												
C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	STATISTICAL TEST
6	7	8	3	4	7	10	7	2	1	0.080519	0.982	Frequency
6	8	4	5	5	7	4	4	7	5	0.897763	0.982	BlockFrequency
8	4	7	4	5	8	5	7	5	2	0.595549	1.000	CumulativeSums
5	7	4	6	5	4	5	6	6	7	0.978072	0.982	CumulativeSums
8	4	4	5	4	4	8	5	9	4	0.554420	0.982	Runs
4	11	4	2	6	6	4	4	5	9	0.145326	1.000	LongestRun
4	4	7	3	4	3	11	6	6	7	0.249284	0.964	Rank
7	6	2	4	5	4	7	8	4	8	0.554420	1.000	FFT
3	4	5	6	11	5	9	5	4	3	0.181557	1.000	NonOverlappingTemplate
6	3	6	6	4	2	5	5	12	6	0.145326	0.964	NonOverlappingTemplate
11	6	4	4	6	4	4	9	4	3	0.181557	0.982	NonOverlappingTemplate
9	4	5	6	4	3	8	4	8	4	0.474986	1.000	NonOverlappingTemplate
1	4	6	10	6	8	7	6	4	3	0.181557	1.000	NonOverlappingTemplate
7	8	6	5	2	5	7	5	6	4	0.759756	1.000	NonOverlappingTemplate
10	4	6	5	8	6	5	4	4	3	0.474986	1.000	NonOverlappingTemplate
6	7	4	8	2	3	4	8	7	6	0.474986	1.000	NonOverlappingTemplate
5	3	4	4	4	7	10	6	7	5	0.514124	0.982	NonOverlappingTemplate
4	4	5	6	6	3	3	8	5	11	0.249284	0.964	NonOverlappingTemplate
3	6	5	6	9	6	4	8	2	6	0.474986	1.000	NonOverlappingTemplate
4	10	1	7	8	6	2	5	6	6	0.145326	1.000	NonOverlappingTemplate
4	7	2	4	8	10	7	4	7	2	0.145326	1.000	NonOverlappingTemplate
7	10	5	2	2	4	3	9	7	6	0.102526	1.000	NonOverlappingTemplate
4	3	7	5	3	7	9	9	3	5	0.304126	1.000	NonOverlappingTemplate
6	5	7	6	8	3	4	8	2	6	0.554420	0.982	NonOverlappingTemplate
4	7	8	7	5	7	4	4	4	5	0.834308	1.000	NonOverlappingTemplate
4	4	8	8	3	6	4	5	10	3	0.275709	1.000	NonOverlappingTemplate
8	4	10	2	7	7	4	1	10	2	0.014550	1.000	NonOverlappingTemplate
10	6	6	5	5	5	4	4	2	8	0.401199	1.000	NonOverlappingTemplate
4	9	2	8	3	7	6	4	5	7	0.366918	1.000	NonOverlappingTemplate
6	6	5	4	5	8	5	6	5	5	0.978072	1.000	NonOverlappingTemplate
4	6	5	8	12	3	2	3	6	6	0.071177	0.964	NonOverlappingTemplate
1	8	7	7	3	4	7	8	5	5	0.334538	1.000	NonOverlappingTemplate
5	9	5	6	6	4	5	2	10	3	0.249284	1.000	NonOverlappingTemplate
4	5	2	6	5	7	7	4	10	5	0.437274	1.000	NonOverlappingTemplate
6	3	11	3	4	7	6	6	6	3	0.249284	0.982	NonOverlappingTemplate

Рисунок 2.6 – Приклад результату тестування NIST

Для зручності аналізу результатів, за допомогою Python-скрипта сформовано Excel-файли для кожної системи у форматі, наведеному на рисунку 2.7. Крім вищевказаних показників в таблицю додано поля PASS_P_V для відображення P_VALUE, які не пройшли порогове значення, PASS_PR, чи пройшла вся бітова послідовність даний тест.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
#	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	PASS P_V	PASS PR?	STATISTICAL TEST
1	4	5	5	4	8	6	3	4	5	11	0,304126	0,982	PASS	PASS	Frequency
2	9	5	6	4	3	7	3	7	4	7	0,554420	0,982	PASS	PASS	BlockFrequency
3	3	8	6	7	5	6	7	7	3	3	0,637119	0,982	PASS	PASS	CumulativeSums
4	4	2	6	11	7	1	8	6	8	2	0,025193	0,982	PASS	PASS	CumulativeSums
5	7	8	5	5	3	5	5	10	3	4	0,401199	0,982	PASS	PASS	Runs
6	5	3	4	6	12	3	7	5	3	7	0,115387	1,000	PASS	PASS	LongestRun
7	7	5	2	6	4	5	9	8	6	3	0,437274	0,982	PASS	PASS	Rank
8	4	7	5	8	4	7	9	6	4	1	0,304126	0,982	PASS	PASS	FFT
9	7	1	6	3	8	5	7	8	4	6	0,366918	1,000	PASS	PASS	NonOverlappingTemplate
10	11	6	6	2	4	6	1	6	10	3	0,025193	1,000	PASS	PASS	NonOverlappingTemplate
11	3	5	4	9	4	5	11	3	3	8	0,090936	1,000	PASS	PASS	NonOverlappingTemplate
12	5	3	6	7	7	6	7	7	2	5	0,719747	1,000	PASS	PASS	NonOverlappingTemplate
13	2	5	5	7	9	4	8	6	8	1	0,162606	0,982	PASS	PASS	NonOverlappingTemplate
14	7	9	5	12	4	2	4	3	2	7	0,021999	1,000	PASS	PASS	NonOverlappingTemplate
15	6	5	4	6	5	4	6	9	6	4	0,867692	0,964	PASS	PASS	NonOverlappingTemplate
16	4	3	8	3	8	5	6	4	6	8	0,554420	1,000	PASS	PASS	NonOverlappingTemplate
17	3	7	4	5	8	4	6	5	4	9	0,595549	1,000	PASS	PASS	NonOverlappingTemplate
18	5	4	3	10	7	1	7	2	10	6	0,037566	0,982	PASS	PASS	NonOverlappingTemplate
19	5	7	11	5	7	6	4	2	5	3	0,224821	1,000	PASS	PASS	NonOverlappingTemplate
20	9	2	5	4	9	3	4	5	6	8	0,249284	1,000	PASS	PASS	NonOverlappingTemplate
21	4	8	8	6	4	7	4	3	6	5	0,719747	1,000	PASS	PASS	NonOverlappingTemplate
22	5	3	4	4	8	3	9	11	4	4	0,102526	0,982	PASS	PASS	NonOverlappingTemplate
23	8	4	7	6	4	4	5	5	8	4	0,798139	0,964	PASS	PASS	NonOverlappingTemplate
24	3	4	7	5	8	6	4	5	6	7	0,834308	1,000	PASS	PASS	NonOverlappingTemplate
25	6	4	9	5	8	4	4	4	6	5	0,719747	1,000	PASS	PASS	NonOverlappingTemplate

Рисунок 2.7 – Приклад результату NIST в excel форматі

Результати тестування всіх генераторів за всіма чотирма режимами зведені за допомогою скриптів до відповідних узагальнених таблиць. Вони містять такі стовпці:

- назву генератора;
- кількість успішно пройдених тестів за критерієм PROPORTION відносно їх загальної кількості;
- кількість успішно пройдених тестів за критерієм p-value;
- кількість тестів, які одночасно пройшли обидва критерії;
- стовпець Low P Value відображає кількість тестів із допустимими значеннями p-value, що знаходяться поблизу граничного порогу, тобто мають значення більше 0,001, але менше 0,01;
- фінальний score, що розраховувався за формулою 2.7.

$$Score = 0,5 \cdot Passed_Rate_P_Value + 0,5 \cdot Pass_Rate_Proportion \quad (2.7)$$

Зведені результати тестування для режиму з різними початковими станами та без хешування (Float/multiple seeds) показано на рисунку 2.8.

Generator	Passed Proportion	Passed P_Value	Passed Total	Low P Value	Score
Logistic-Lorenz	188/188	188/188	188/188	11/188	1,000
Logistic-Rossler	188/188	186/188	186/188	4/188	0,995
Logistic-Chua	188/188	187/188	187/188	4/188	0,997
Logistic-Duffing	188/188	188/188	188/188	7/188	1,000
Logistic-Van der Pol	188/188	188/188	188/188	3/188	1,000
Logistic-Forced pen	188/188	188/188	188/188	5/188	1,000

Рисунок 2.8 – Зведені результати Float/multiple seeds

Для режиму хешування для послідовностей з різними початковими станами (SHA-256/multiple seeds) одержані наступні результати тестування, які відображені на рисунку 2.9.

Generator	Passed Proportion	Passed P_Value	Passed Total	Low P Value	Score
Logistic-Lorenz	186/188	187/188	186/188	0/188	0,989
Logistic-Rossler	187/188	185/188	184/188	8/188	0,979
Logistic-Chua	188/188	188/188	188/188	3/188	1,000
Logistic-Duffing	188/188	187/188	187/188	2/188	0,995
Logistic-Van der Pol	187/188	186/188	185/188	4/188	0,984
Logistic-Forced pen	188/188	187/188	187/188	6/188	0,995

Рисунок 2.9 – Зведені результати SHA-256/multiple seeds

Зведені результати тестування для режиму з фіксованим початковим станом та без хешування (Float/one seed) показано на рисунку 2.10.

Generator	Passed Proportion	Passed P_Value	Passed Total	Low P Value	Score
Logistic-Lorenz	188/188	188/188	188/188	4/188	1,000
Logistic-Rossler	188/188	188/188	188/188	5/188	1,000
Logistic-Chua	187/188	188/188	187/188	3/188	0,997
Logistic-Duffing	187/188	186/188	186/188	3/188	0,992
Logistic-Van der Pol	186/188	186/188	184/188	6/188	0,989
Logistic-Forced pen	187/188	185/188	185/188	9/188	0,989

Рисунок 2.10 – Зведені результати Float/one seed

Для режиму хешування для послідовностей з постійним початковим станом (SHA-256/one seed) одержані наступні результати тестування, які відображені на рисунку 2.11

Generator	Passed Proportion	Passed P_Value	Passed Total	Low P Value	Score
Logistic-Lorenz	186/188	188/188	186/188	3/188	0,989
Logistic-Rossler	188/188	187/188	187/188	3/188	0,995
Logistic-Chua	188/188	188/188	188/188	5/188	1,000
Logistic-Duffing	188/188	187/188	187/188	3/188	0,995
Logistic-Van der Pol	186/188	186/188	184/188	1/188	0,979
Logistic-Forced pen	187/188	187/188	186/188	3/188	0,989

Рисунок 2.11 – Зведені результати SHA-256/one seed

Для подальшого аналізу отриманих результатів здійснено порівняння систем за початковими станами one seed та multiple seeds у розрізі режимів Float та SHA-256. Порівняння one seed та multiple seeds показує, наскільки статистична якість залежить від способу вибору початкових умов.

У режимі Float зведені результати відображено у таблиці 2.2.

Таблиця 2.2 – Порівняння результатів Float для one seed та multiple seeds

Генератор	Score OS	Score MS	Passed Total OS	Passed Total MS	Passed Total MS-OS	Low P OS	Low P MS
Logistic-Lorenz	1,000	1,000	188	188	0	4	11
Logistic-Rossler	1,000	0,995	188	186	-2	5	4
Logistic-Chua	0,997	0,997	187	187	0	3	4
Logistic-Duffing	0,992	1,000	186	188	2	3	7
Logistic-Van der Pol	0,989	1,000	184	188	4	6	3
Logistic-Forced Pendulum	0,989	1,000	185	188	3	9	5

Для наочного порівняння дані відтворено у вигляді гістограми на рисунку 2.12.

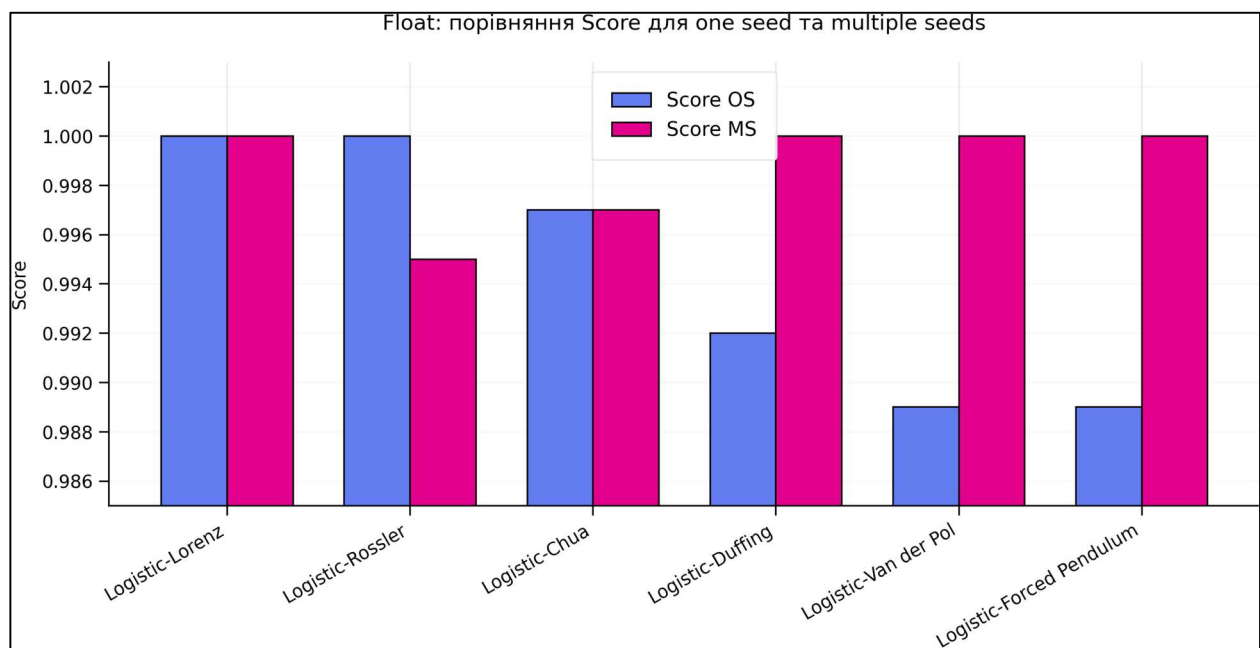


Рисунок 2.12 – Гістограма для Float: one seed та multiple seeds

У Float-режимі Logistic-Lorenz демонструє максимальну стабільність: Score має 1,000 і Passed Total демонструє 188/188 для обох seed-режимів. Logistic-Rossler має максимальний результат у one seed, але Score знижується до 0,995 у multiple seeds. У Logistic-Chua значення Score дорівнює 0,997 в обох режимах.

Найбільше покращення від використання режиму multiple seeds спостерігається для Logistic-Van der Pol, Logistic-Forced Pendulum та Logistic-Duffing. Для Logistic-Van der Pol Passed Total зростає з 184 до 188, для Logistic-Forced Pendulum – з 185 до 188, а для Logistic-Duffing – з 186 до 188.

У режимі SHA-256 зведені результати відображено у таблиці 2.3. У цьому режимі порівнюється, як змінюється результат при переході від одного seed до multiple seeds. Таким чином, можна з'ясувати чи дає SHA-256 можливість компенсувати можливі статистичні недоліки первинної хаотичної послідовності.

Таблиця 2.3 – Порівняння результатів SHA для one seed та multiple seeds

Генератор	Score OS	Score MS	Passed Total OS	Passed Total MS	Passed Total MS-OS	Low P OS	Low P MS
Logistic-Lorenz	0,989	0,989	186	186	0	3	0
Logistic-Rossler	0,995	0,979	187	184	-3	3	8
Logistic-Chua	1,000	1,000	188	188	0	5	0
Logistic-Duffing	0,995	0,995	187	187	0	3	0
Logistic-Van der Pol	0,979	0,984	184	185	1	1	0
Logistic-Forced Pendulum	0,989	0,995	186	187	1	3	0

Для наочного порівняння дані відтворено у вигляді гістограми на рисунку 2.13.

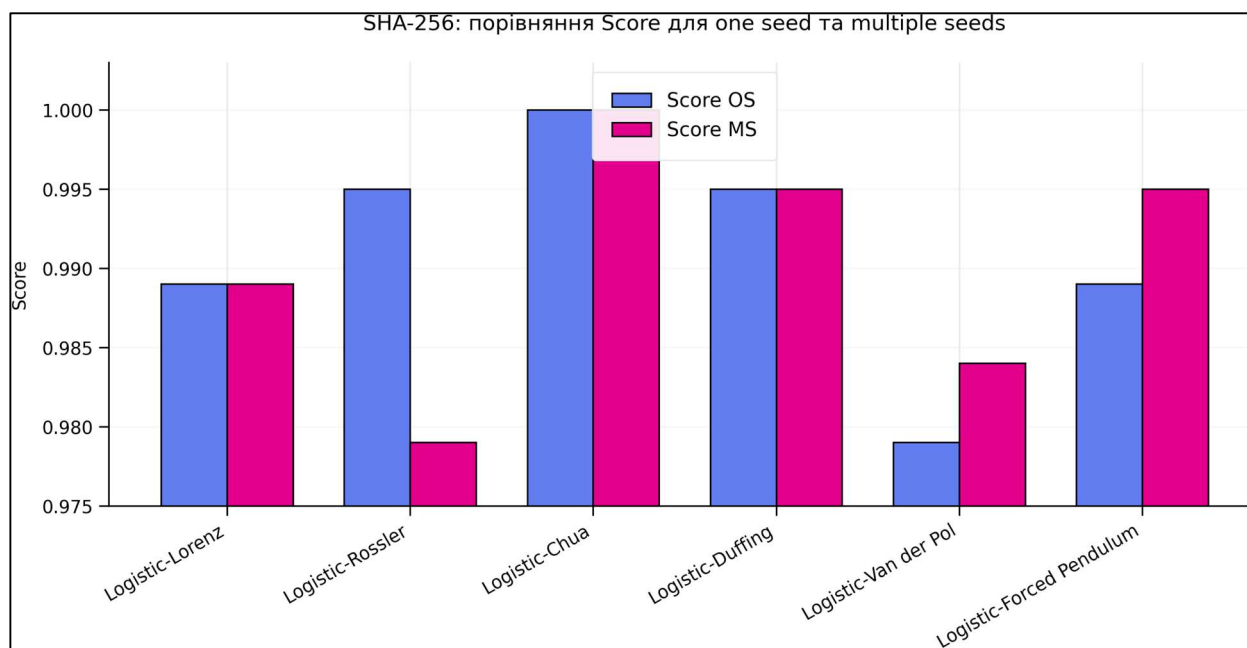


Рисунок 2.13 – Гістограма для SHA-256: one seed та multiple seeds

У SHA-256 режимі найкращий результат має Logistic-Chua зі значенням Score, який дорівнює 1,000 та Passed Total, який демонструє 188/188 як для one seed, так і для multiple seeds. Це може свідчити про найбільш стабільну статистичну якість цієї системи саме у режимі хешування.

Logistic-Duffing демонструє стабільний результат зі значенням Score 0,995 в обох seed-режимах. Logistic-Forced Pendulum покращується з 0,989 до 0,995 при переході до multiple seeds. Logistic-Van der Pol також дещо покращується з 0,979 до 0,984, але залишається нижче за лідерів. Logistic-Rosler, навпаки, знижується з 0,995 до 0,979, що вказує на більшу чутливість до зміни початкових станів у SHA-256 режимі.

Для узагальненого одночасного оцінювання генераторів у двох способах формування бітової послідовності використано інтегральний показник Float/SHA-256. Він може свідчити, наскільки генератор зберігає статистичну стійкість незалежно від способу формування бітової послідовності – шляхом прямого перетворення числових значень у біти або із застосуванням хеш-постобробки SHA-256.

Інтегральний показник Float/SHA-256 залежно від заданого seed-режиму розраховувався за формулами 2.8 та 2.9:

$$I_{Float/SHA(OS)} = 0,5 \cdot Score_{Float(OS)} + 0,5 \cdot Score_{SHA - 256(OS)} \quad (2.8)$$

$$I_{Float/SHA(MS)} = 0,5 \cdot Score_{Float(MS)} + 0,5 \cdot Score_{SHA - 256(MS)} \quad (2.9)$$

Значення показника знаходиться у діапазоні [0; 1]. Чим ближче значення до 1, тим стабільніше генератор проходить NIST STS одночасно для Float та SHA-256. Зведені результати відображено у таблиці 2.4.

Таблиця 2.4 – Порівняння Float/SHA-256 для one seed та multiple seeds

Генератор	Float OS	SHA-256 OS	I Float/SHA OS	Float MS	SHA-256 MS	I Float/SHA MS	I MS-OS
Logistic-Lorenz	1,000	0,989	0,994	1,000	0,989	0,994	0,000
Logistic-Rossler	1,000	0,995	0,998	0,995	0,979	0,987	-0,0105
Logistic-Chua	0,997	1,000	0,998	0,997	1,000	0,998	0,000
Logistic-Duffing	0,992	0,995	0,994	1,000	0,995	0,998	0,0040
Logistic-Van der Pol	0,989	0,979	0,984	1,00	0,984	0,992	0,0080
Logistic-Forced Pendulum	0,989	0,989	0,989	1,000	0,995	0,998	0,0085

Для наочного порівняння дані відтворено у вигляді гістограми на рисунку 2.14.

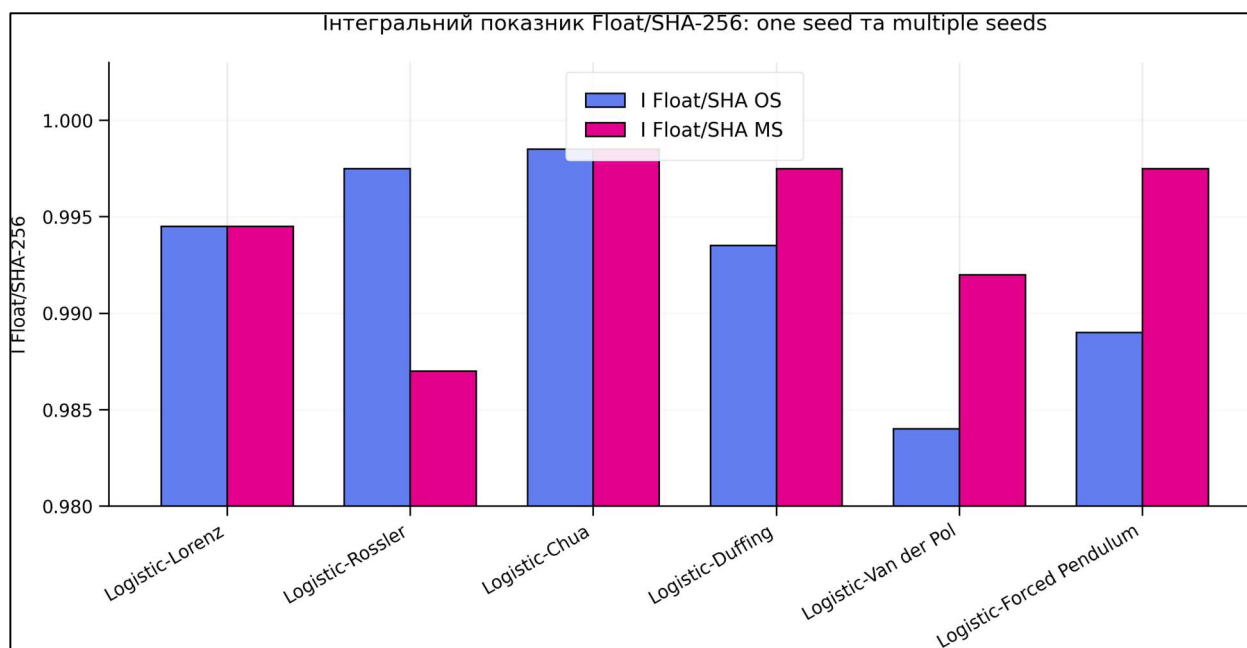


Рисунок 2.14 – Гістограма для I_Float/SHA-256: one seed та multiple

Згідно з одержаними результатами, у режимі one seed найкращими є Logistic-Chua, Logistic-Rossler та Logistic-Lorenz. Logistic-Chua має високий Float Score і максимальний SHA-256 Score, тому отримує одне з найкращих інтегральних значень. Logistic-Rossler має сильний для one seed результат, але його стійкість погіршується у режимі multiple seeds.

У режимі multiple seeds найкращі інтегральні значення мають Logistic-Chua, Logistic-Duffing та Logistic-Forced Pendulum. Для Logistic-Duffing, Logistic-Van der Pol і Logistic-Forced Pendulum інтегральний показник зростає при переході до multiple seeds. Це означає, що використання різних початкових станів покращує їхню узагальнену статистичну якість з урахуванням і Float, і SHA-256.

Logistic-Rossler є основним винятком: його інтегральний показник зменшується з 0,9975 до 0,9870. Таким чином, для цієї системи multiple seeds не дає покращення за сукупністю Float/SHA-256, хоча результат залишається високим.

Показник I_Float/SHA(OS) характеризує генератор лише для фіксованого початкового стану, а I_Float/SHA(MS) — лише для різних початкових станів. Для підсумкового порівняння систем можна використати загальний інтегральний

показник, який одночасно враховує всі чотири результати: Float one seed, Float multiple seeds, SHA-256 one seed та SHA-256 multiple seeds.

Загальний інтегральний показник Float/SHA-256/OS/MS визначається як середнє арифметичне чотирьох Score:

$$I_{Float/SHA/OS/MS} = 0,5 \cdot I_{Float/SHA(OS)} + 0,5 \cdot I_{Float/SHA(MS)} \quad (2.10)$$

Таким чином, новий показник оцінює не окремий режим, а загальну статистичну стабільність генератора за всіма способами формування послідовності та ініціалізації. Зведені результати відображено у таблиці 2.5, системи розташовані у таблиці за рейтингом.

Таблиця 2.5 – Загальний показник Float/SHA-256/OS/MS для всіх режимів

Генератор	Float OS	Float MS	SHA-256 OS	SHA-256 MS	I Float/SHA OS	I Float/SHA MS	I Float/SHA/OS/MS
Logistic-Chua	0,997	0,997	1,000	1,000	0,998	0,998	0,998
Logistic-Duffing	0,992	1,000	0,995	0,995	0,994	0,998	0,996
Logistic-Lorenz	1,000	1,000	0,989	0,989	0,994	0,994	0,994
Logistic-Forced Pendulum	0,989	1,000	0,989	0,995	0,989	0,998	0,993
Logistic-Rossler	1,000	0,995	0,995	0,979	0,998	0,987	0,992
Logistic-Van der Pol	0,989	1,000	0,979	0,984	0,984	0,992	0,988

Для наочного порівняння дані відтворено у вигляді гістограми на рисунку 2.15.

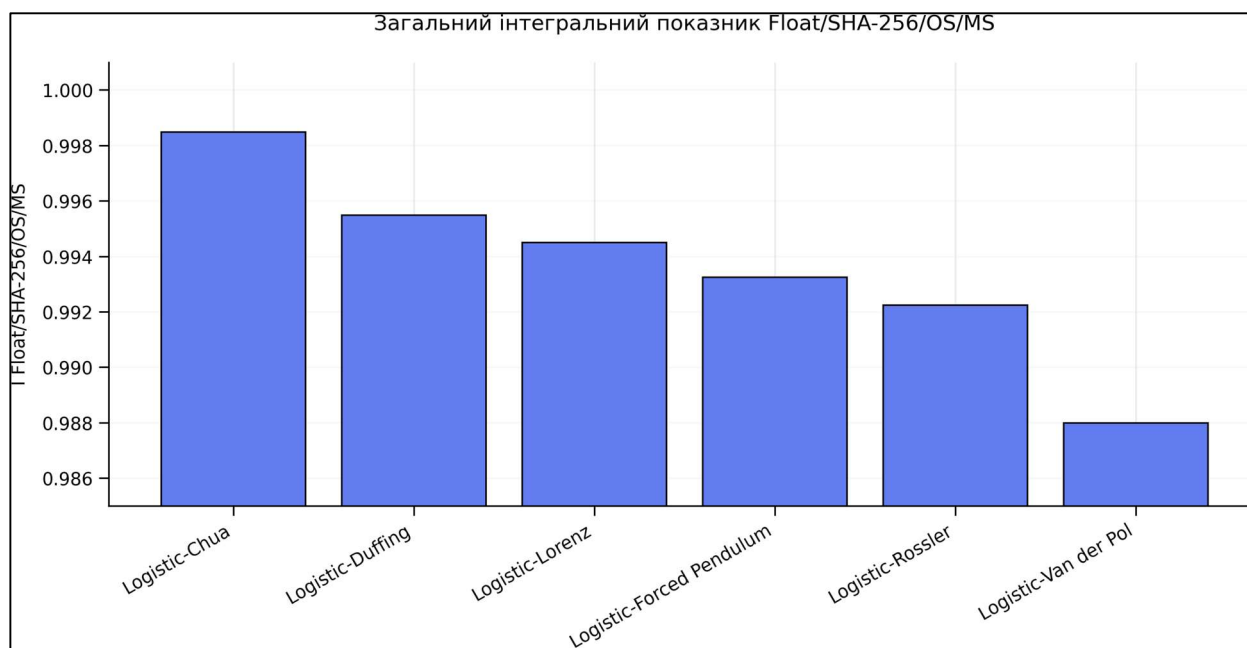


Рисунок 2.15 – Гістограма загального інтегрального показника

За загальним інтегральним показником найкращий результат має Logistic-Chua – 0,9985, у системи максимальний результат у режимі SHA-256 для обох seed-режимів і стабільно високий результат у Float. Другу позицію займає Logistic-Duffing – 0,9955, оскільки multiple seeds повністю компенсує слабший Float one seed результат, а SHA-256 залишається стабільним.

Logistic-Lorenz і Logistic-Forced Pendulum мають близькі значення: 0,9945 та 0,99325 відповідно. Logistic-Lorenz є більш рівномірним за режимами, тоді як Logistic-Forced Pendulum виграє за рахунок покращення у режимі multiple seeds. Logistic-Rossler має 0,99225 через зниження результатів у multiple seeds, а Logistic-Van der Pol має найнижче серед розглянутих систем значення 0,9880, що пов'язано зі слабшим SHA-256 one seed результатом порівняно з іншими системами.

2.3.2 Тестування розроблених систем на швидкодії

Окрім статистичної стійкості, проведено тестування швидкодії генераторів. Для цього використовувалися два режими: Float та SHA-256. Результати тестування показано на рисунку 2.16.

Chaotic PRNG — One Seed ~6.875 MB/run 3 repeats			
System	Mode	Speed (MB/s)	Avg time (ms)
Logistic-Lorenz	Float	0,674	9732,8
Logistic-Lorenz	SHA-256	3,922	1671,8
Logistic-Rossler	Float	1,162	5640,8
Logistic-Rossler	SHA-256	2,625	2498,1
Logistic-Chua	Float	0,545	12023,1
Logistic-Chua	SHA-256	2,016	3253,0
Logistic-Duffing	Float	0,656	9996,8
Logistic-Duffing	SHA-256	4,073	1609,7
Logistic-Van der Pol	Float	0,858	7645,6
Logistic-Van der Pol	SHA-256	2,245	2920,8
Logistic-Forced Pendulum	Float	0,875	7488,9
Logistic-Forced Pendulum	SHA-256	4,105	1597,2

Рисунок 2.16 – Результати benchmark-тестування генераторів

Порівняння результатів тестування наведено у таблиці 2.6.

Таблиця 2.6 – Порівняння швидкодії Float та SHA-256

Генератор	Float, MB/s	SHA-256, MB/s	Прискорення SHA/Float
Logistic-Lorenz	0,674	3,922	5,819
Logistic-Rossler	1,162	2,625	2,259
Logistic-Chua	0,545	2,016	3,699
Logistic-Duffing	0,656	4,073	6,209
Logistic-Van der Pol	0,858	2,245	2,617
Logistic-Forced Pendulum	0,875	4,105	4,691

Для наочного порівняння дані відтворено у вигляді гістограми на рисунку 2.17.

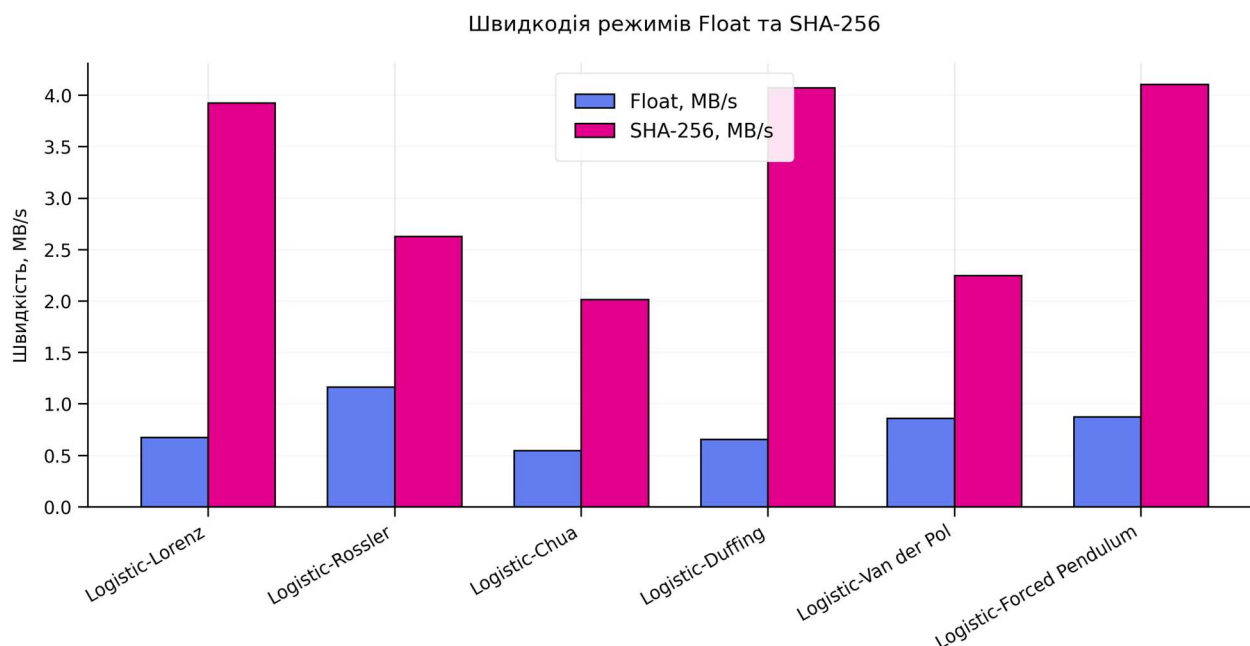


Рисунок 2.17 – Гістограма швидкодії Float та SHA-256

Результати benchmark-тестування показують, що SHA-256 режим у реалізованому програмному середовищі є швидшим за Float для всіх досліджених систем. Найвищу швидкість має Logistic-Forced Pendulum SHA-256 – 4,105 MB/s. Дуже близький результат має Logistic-Duffing SHA-256 – 4,073 MB/s. Третє місце займає Logistic-Lorenz SHA-256 – 3,922 MB/s. Найбільше відносне прискорення SHA-256 відносно Float спостерігається для Logistic-Duffing — приблизно 6,21 раза. Для Logistic-Lorenz прискорення становить приблизно 5,82 раза, для Logistic-Forced Pendulum — приблизно 4,69 раза

2.3.3 Порівняльний аналіз одержаних результатів тестування розроблених систем

Для фінального рейтингу використано комплексний індекс (формули 2.11 та 2.12), який враховує загальний інтегральний показник Float/SHA-256/OS/MS та нормовану швидкість SHA-256. Статистична складова має вагу 0,7, а швидкісна – 0,3, оскільки статистична придатність генератора є пріоритетною.

$$K = 0,7 \cdot I_Float/SHA/OS/MS + 0,3 \cdot V_SHA_norm \quad (2.11)$$

де

$$V_SHA_norm = V_SHA / \max(V_SHA) \quad (2.12)$$

Результати комплексного рейтингу за всіма метриками зведено до таблиці 2.7.

Таблиця 2.7 – Комплексний рейтинг систем

Місце	Генератор	I Float/SHA/ OS/MS	Нормована швидкість SHA	Комплексний індекс
1	Logistic-Forced Pendulum	0,993	1,000	0,995
2	Logistic-Duffing	0,996	0,992	0,995
3	Logistic-Lorenz	0,994	0,955	0,983
4	Logistic-Rossler	0,992	0,639	0,886
5	Logistic-Van der Pol	0,988	0,547	0,856
6	Logistic-Chua	0,998	0,491	0,846

Порівняння за загальним інтегральним показником показало, що найкращу сумарну статистичну якість має Logistic-Chua. Logistic-Duffing, Logistic-Lorenz і Logistic-Forced Pendulum також демонструють високий рівень стабільності. Logistic-Rossler має зниження через слабші результати у multiple seeds, а Logistic-Van der Pol поступається через нижчий SHA-256 one seed результат.

З урахуванням швидкодії найбільш перспективними для практичного застосування є Logistic-Forced Pendulum SHA-256 та Logistic-Duffing SHA-256. Якщо головним критерієм є саме статистична стабільність за всіма режимами, доцільно виділити Logistic-Chua, Logistic-Duffing і Logistic-Lorenz.

За комплексним індексом найкращими є Logistic-Forced Pendulum, Logistic-Duffing та Logistic-Lorenz. Logistic-Chua має найкращий загальний інтегральний показник, але поступається за швидкістю SHA-256, тому в комплексному рейтингу розташовується нижче за швидші системи.

Гібридні хаотичні системи підтвердили придатність для генерації псевдовипадкових послідовностей. З іншого боку, окремий аналіз Float та SHA-256 показав, що результати можуть відрізнятися залежно від способу формування бітів, тому оцінювання лише одного режиму є недостатнім.

Загальний інтегральний показник Float/SHA-256/OS/MS дозволив оцінити генератор за всіма дослідженими режимами одночасно. Такий підхід вважаю більш повним, оскільки він враховує і спосіб формування бітів, і вплив початкових станів.

Висновки до розділу 2

У цьому розділі виконано математичне моделювання та алгоритмізацію гібридних систем шифрування на основі детермінованого хаосу. Розглянуто аналітичні моделі логістичного відображення, систем Лоренца, Ресслера, Чуа, осциляторів Дуффінга та Ван дер Поля, а також маятника з силою, що змушує. Для неперервних систем описано застосування чисельного методу Рунге–Кутта четвертого порядку, що дозволяє реалізувати їхню динаміку у програмному середовищі.

На основі поєднання дискретного логістичного відображення з неперервними хаотичними системами розроблено алгоритми генерації псевдовипадкових послідовностей, які використовуються як ключовий потік у процесах шифрування. Також описано алгоритми шифрування та розшифрування тексту, зображень, аудіоінформації та файлів із використанням операцій XOR, афінних перетворень, фрактальних сортувальних матриць, дифузійних процедур і SHA-256.

Проведені статистичне тестування за допомогою NIST STS та аналіз швидкодії підтвердили придатність розроблених гібридних генераторів для криптографічних задач. Найкращі результати за сукупністю статистичної якості та продуктивності показали комбінації Logistic-Forced Pendulum, Logistic-Duffing, Logistic-Lorenz і Logistic-Chua.

РОЗДІЛ 3. Програмна реалізація та тестування застосунку

3.1 Обґрунтування вибору засобів та середовища розробки

Веб-додаток було розроблено на основі хаотичних алгоритмів, а також з використанням мов програмування Python, JavaScript і фреймворку FastAPI у середовищі Visual Studio для шифрування текстової інформації, зображень, аудіо та файлів. Обраний інструментарій забезпечує продуктивну серверну обробку, точне чисельне моделювання та зручну взаємодію з користувачем у браузері, що є важливим для демонстрації роботи шифрів за різних форматів даних.

Python обрано як інструмент розробки логіки криптографічних систем на основі хаотичних алгоритмів, оскільки він забезпечує зручне та швидке чисельне моделювання нелінійних динамічних систем. Додатковими перевагами Python є простота роботи з масивами даних, файлами, зображеннями та аудіоданими, а також наявність великої кількості прикладних бібліотек. Крім того, у разі виникнення потреби у високих обчислювальних потужностях для реалізації складних алгоритмів, Python дає можливість інтеграції з компонентами C/C++, що дозволяє переносити найбільш критичні за швидкістю частини у компільований код.

FastAPI обрано як основний фреймворк для розробки програмного інтерфейсу додатку шифрування, оскільки він поєднує високу продуктивність із низькими затримками під час обробки запитів, що є важливим для задач генерації псевдовипадкових послідовностей, а також для оперативного виконання процедур шифрування й розшифрування. Також перевагою фреймворку є підтримка асинхронної моделі, завдяки якій сервер може ефективно опрацьовувати паралельні звернення без блокування обробки запитів. А завдяки вбудованій типізації та автоматичній генерації документації інтерфейс фреймворку є чітким та однозначним і формалізованим, при цьому інтеграція з клієнтською частиною є нескладною та зручною. Крім того, FastAPI просто та ефективно здійснює передавання файлів різних типів, що повністю відповідає

потребам демонстраційного програмного забезпечення. У підсумку функціонал фреймворку дозволяє відокремити обчислювальне ядро моделювання хаосу від транспортного рівня API, що дозволяє сформувати модульну структуру для подальшого масштабування за потреби та підтримки проєкту.

Завдяки сценаріям JavaScript спрощено розгортання та демонстрацію роботи застосунку, що дозволило створити кросбраузерну клієнтську частину. Використання JavaScript дозволило отримати повний контроль над UI-елементами, зокрема формами параметрів, валідацією та сценаріями роботи з файлами. Додатковою перевагою JavaScript є легка інтеграція з бекендом на FastAPI через стандартні HTTP-запити, що дає змогу відстежувати прогрес обробки у реальному часі та відображати результати взаємодії з API без перезавантаження сторінки. Такий підхід мінімізує кількість сторонніх залежностей і знижує загальну складність розробки, залишаючи логіку інтерфейсу контрольованою.

3.2 Загальні вимоги та сценарії використання програмного додатку

Розроблений програмний додаток має демонструвати практичне використання гібридних хаотичних систем у задачах шифрування та розшифрування даних різних типів. Застосунок повинен забезпечувати взаємодію користувача з гібридними хаотичними системами шифрування через вебінтерфейс.

Загальна логіка роботи додатку повинна ґрунтуватися на клієнт-серверній архітектурі: користувач має обирати тип даних, задавати параметри генератора, визначати операцію шифрування чи розшифрування та передавати дані на обробку, а серверна частина — готувати вхідні дані до внутрішнього представлення, формувати псевдовипадкову послідовність на основі обраної хаотичної системи, виконувати криптографічне перетворення та повертати результат клієнтській частині.

У межах застосунку повинна бути передбачена робота з текстовими даними, зображеннями, аудіофайлами та файлами загального призначення. Текстові дані мають вводитися у відповідне поле інтерфейсу, а зображення, аудіо та інші файли — завантажуватися з локального пристрою користувача. Спеціалізована обробка аудіоданих має забезпечуватися для WAV-файлів, а спеціалізована обробка зображень — для PNG-файлів; інші аудіоформати та формати зображень можуть оброблятися як файли загального призначення.

Детальний опис вхідних даних, функціональних і нефункціональних вимог, а також сценаріїв взаємодії користувача із системою наведено у наступних підрозділах.

3.2.1 Опис вхідних даних

Вхідні дані програмного додатку поділяються на дві основні групи: дані, що підлягають шифруванню або розшифруванню, та керуючі параметри, які визначають режим роботи алгоритмів і початкові умови генератора псевдовипадкових послідовностей.

До першої групи належать такі типи даних:

- текстові дані – повідомлення, що вводиться користувачем у відповідне поле вебінтерфейсу. Текст може оброблятися як послідовність символів або як байтова послідовність залежно від обраного режиму роботи;

- графічні дані – зображення, що завантажуються користувачем з локального пристрою і подаються у вигляді матриці пікселів. Основним форматом для демонстрації роботи алгоритму є PNG, оскільки він забезпечує збереження графічної інформації без втрат;

- аудіодані – аудіофайли, що завантажуються через інтерфейс користувача. Для спеціалізованої обробки використовується формат WAV, оскільки він зберігає звукові дані без стиснення. Інші аудіоформати можуть розглядатися як файли загального призначення;

– файли загального призначення – довільні файли незалежно від їхнього розширення та внутрішньої структури. В цьому випадку файл розглядається як байтова послідовність.

Внутрішнє подання вхідних даних залежить від типу: текстові дані подаються у вигляді символів або байтів, зображення – у вигляді матриці пікселів, аудіофайли – у вигляді байтового потоку, а файли загального призначення – як послідовність байтів.

Другою групою вхідних даних є параметри, що задаються користувачем для виконання криптографічного перетворення. До них належать:

- тип операції: шифрування або розшифрування;
- тип оброблюваних даних: текст, зображення, аудіо або файл;
- режим роботи алгоритму для текстових даних;
- вибір гібридної хаотичної системи;
- початкове значення логістичного відображення;
- початкові умови відповідної неперервної хаотичної системи;
- параметри, необхідні для відтворення тієї самої псевдовипадкової послідовності під час розшифрування.

Початкові умови хаотичних систем повинні належати до допустимих діапазонів, визначених під час моделювання генераторів. Дотримання цих обмежень є необхідним для коректної роботи алгоритмів, оскільки однакові початкові параметри мають забезпечувати відтворення однакової псевдовипадкової послідовності під час шифрування та розшифрування.

3.2.2 Функціональні та нефункціональні вимоги

Функціональні вимоги визначають набір можливостей, які повинен забезпечувати програмний додаток. З урахуванням призначення системи до основних функціональних вимог належать:

- система повинна забезпечувати введення текстових даних та завантаження файлів зображень, аудіофайлів і файлів довільного формату;
- застосунок має підтримувати окремі режими роботи з текстом, зображеннями, аудіофайлами та файлами загального призначення;
- система повинна надавати можливість введення параметрів генератора, зокрема початкових умов логістичного відображення та неперервної хаотичної системи;
- програмний додаток повинен перевіряти коректність введених параметрів і не допускати запуску алгоритму за межами допустимих діапазонів;
- серверна частина має генерувати псевдовипадкові числові або байтові послідовності на основі гібридних хаотичних систем;
- система повинна виконувати шифрування та розшифрування тексту з використанням XOR-операції або афінного перетворення;
- застосунок має забезпечувати шифрування та розшифрування зображень шляхом скремблювання, дифузії та подальшого відновлення графічного файлу;
- система повинна підтримувати обробку аудіофайлів шляхом перетворення їх у числовий або байтовий масив із подальшим застосуванням XOR-операції;
- додаток має виконувати побайтове шифрування та розшифрування файлів довільного формату без урахування їх внутрішньої структури;
- для зображень та аудіофайлів система повинна забезпечувати попередній перегляд або прослуховування результатів, якщо це технічно можливо;
- користувач повинен мати можливість зберігати результат обробки у вигляді окремого файлу.

Для визначення якісних характеристик програмного продукту та умов його експлуатації сформульовано такі нефункціональні вимоги:

- система повинна забезпечувати достатню продуктивність для демонстраційної обробки тексту, зображень, аудіофайлів і файлів помірної розміру;

- за однакових початкових умов генератор має формувати однаковою псевдовипадкову послідовність, що є необхідною умовою коректного розшифрування;

- після послідовного виконання шифрування та розшифрування початкові дані повинні відновлюватися без втрат за умови використання правильних параметрів;

- застосунок повинен бути стійким до помилок користувача: перевіряти введені дані, проводити обробку вхідних даних: перевіряти типи файлів, обмежувати некоректні значення та не виконувати небезпечні операції з даними користувача.

3.2.3 Use Case діаграма

Use Case-діаграма демонстраційного застосунку, показана на рисунку 3.1, відображає основні сценарії взаємодії користувача із системою через вебінтерфейс. Користувач може задавати параметри шифрування або розшифрування, вводити чи завантажувати дані, переглядати файли, запускати обробку, а також переглядати або зберігати отримані результати.

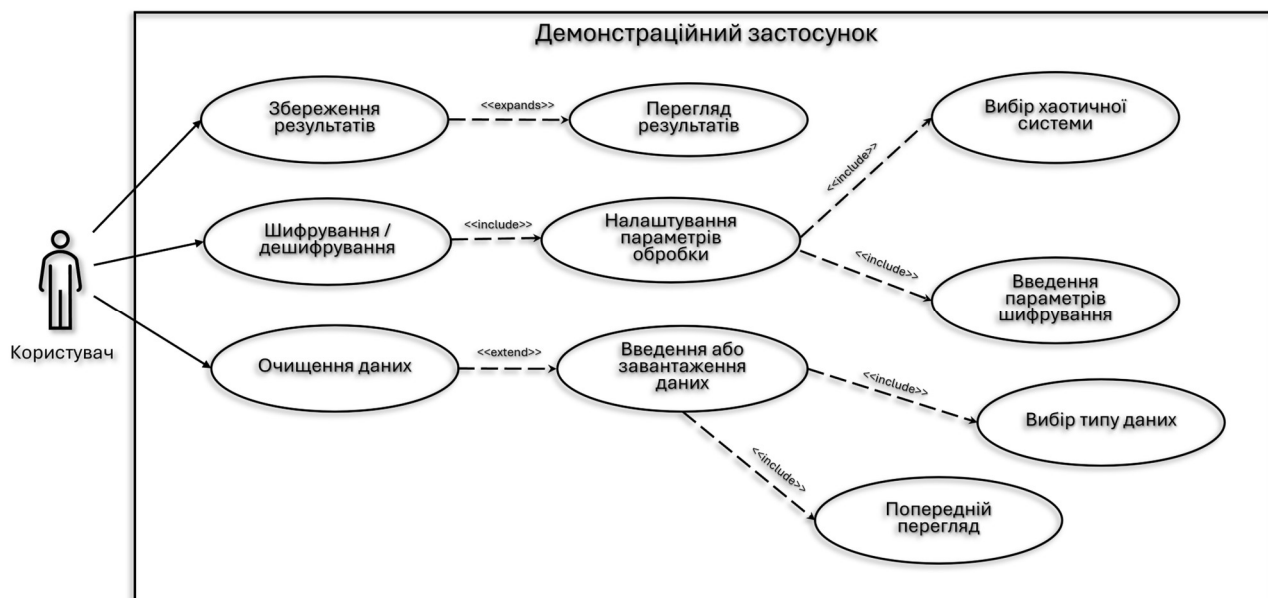


Рисунок 3.1 – Use Case-діаграма демонстраційного застосунку

Головним варіантом використання є «Шифрування / розшифрування», оскільки він реалізує основну функціональність застосунку. Цей процес охоплює підготовку даних, генерацію керувальної псевдовипадкової послідовності, безпосереднє криптографічне перетворення та формування результату у форматі, придатному для перегляду або збереження.

Таким чином, Use Case-діаграма узагальнює функціональні вимоги до застосунку з позиції користувача та відображає шифрування або розшифрування як комплексний процес обробки даних.

3.3 Опис архітектури та графічного інтерфейсу програми

3.3.1 Опис архітектури

Розроблене програмне забезпечення реалізовано як демонстраційний вебзастосунок із клієнт-серверною архітектурою. Така структура обрана для відокремлення інтерфейсу користувача від обчислювальної логіки, пов'язаної з формуванням гібридних хаотичних послідовностей, перетворенням даних та виконанням операцій шифрування і розшифрування.

Таблиця 3.1 – Узагальнена структура програмного забезпечення

Компонент	Реалізація	Призначення
Клієнтський рівень	HTML, CSS, JavaScript	Введення параметрів, завантаження даних, попередній перегляд і збереження результатів.
Серверний рівень	Python, FastAPI	Обробка запитів, координація модулів і виконання основної логіки застосунку.
Конвертери даних	Pillow,, засоби Python	Перетворення тексту, зображень та файлів у внутрішнє подання і назад.
Генератор хаосу	Logistic + неперервні системи, RK4, SHA-256	Формування відтворюваних псевдовипадкових послідовностей для шифрування і тестування.
Криптографічний модуль	XOR, афінне перетворення, FSM, дифузія	Виконання шифрування та розшифрування відповідно до типу даних.

Архітектура застосунку, наведена в таблиці 3.1 та проілюстрована на рисунку 3.2, складається з клієнтського рівня, серверного рівня та набору прикладних модулів обробки даних. Клієнтська частина реалізована засобами HTML, CSS і JavaScript та відповідає за введення параметрів, вибір режиму роботи, завантаження файлів, попередній перегляд результатів і збереження сформованих даних. Серверна частина реалізована мовою Python із використанням FastAPI, що забезпечує приймання запитів від клієнта, маршрутизацію операцій і передавання даних до відповідних модулів обробки.

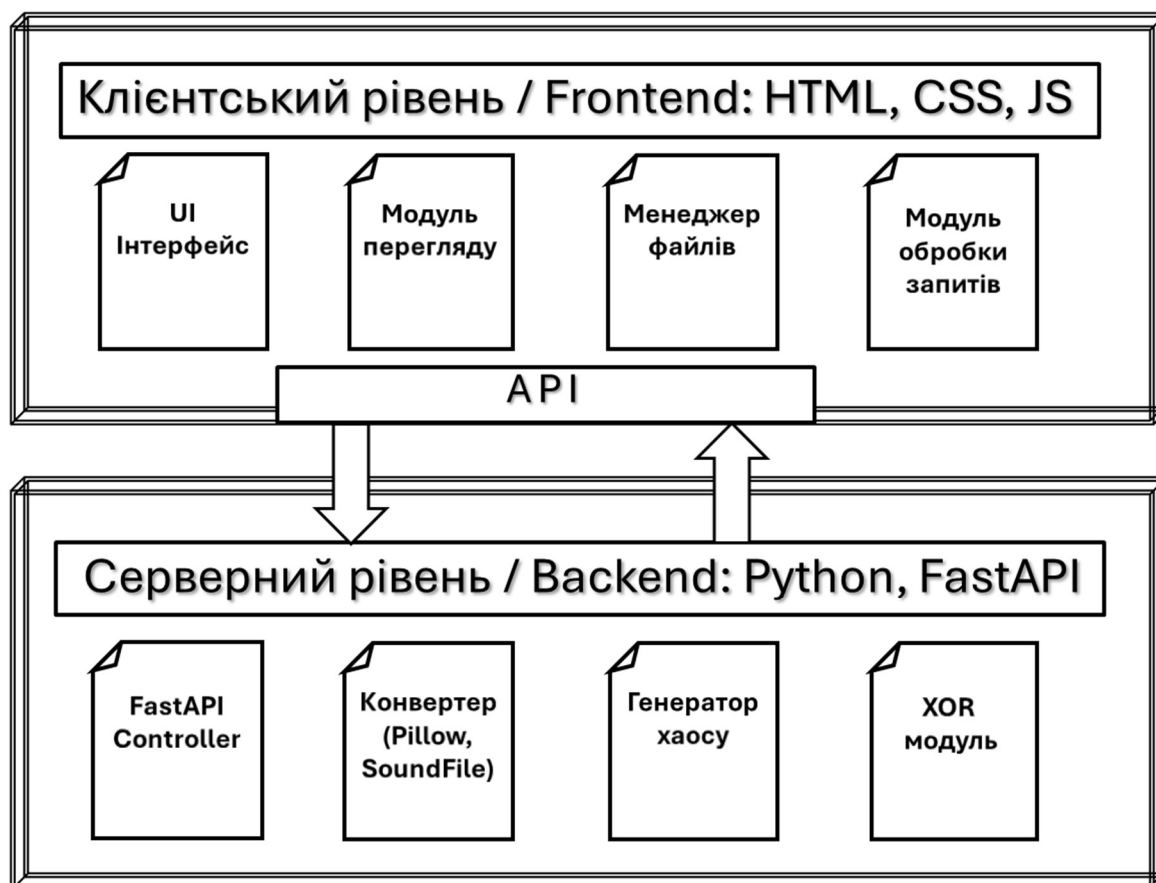


Рисунок 3.2 – Схема архітектури демонстраційного застосунку

Узагальнено взаємодію компонентів можна подати як послідовність: інтерфейс користувача формує запит із параметрами та вхідними даними; API-рівень передає його контролеру; контролер визначає тип даних і необхідний алгоритм; конвертер приводить дані до внутрішнього подання; генератор формує псевдовипадкову послідовність; криптографічний модуль виконує шифрування або розшифрування; після цього результат повертається клієнтській частині для перегляду або збереження.

Центральним серверним компонентом є генератор гібридних хаотичних послідовностей. Його винесення в окремий модуль зумовлене тим, що він використовується не лише у процесі шифрування, а й для формування послідовностей, придатних для подальшого статистичного аналізу. Генератор поєднує дискретне логістичне відображення з однією з неперервних хаотичних систем, стан якої оновлюється чисельним методом Рунге – Кутта четвертого

порядку. Параметри генерації та початкові умови виступають ключовими даними, від яких залежить відтворюваність результату.

Криптографічний модуль використовує сформовані генератором послідовності відповідно до типу вхідних даних. Для тексту передбачено побайтовий режим XOR та посимвольне перетворення. Для файлів загального призначення і WAV-аудіо використовується побайтове XOR-шифрування, оскільки воно не залежить від змісту файла та дозволяє зберегти оборотність операції. Для зображень застосовується окрема схема, що поєднує скремблювання пікселів за допомогою фрактальної сортувальної матриці та дифузії на основі XOR-перетворень. Детальний математичний опис цих алгоритмів наведено у попередніх підрозділах.

Модуль конвертації даних забезпечує перехід між зовнішнім форматом файлу та внутрішнім представленням, необхідним для роботи алгоритмів. Текст перетворюється у байтову або символьну послідовність, файли — у байтовий потік, аудіо — у масив числових значень, а зображення — в RGB-матрицю пікселів. Для роботи з зображеннями використовується Pillow. Після завершення криптографічного перетворення цей самий модуль бере участь у відновленні формату результату.

Окреме значення для архітектури має підтримка дослідницького контуру роботи. Оскільки в межах роботи порівнюються різні гібридні генератори та режими формування послідовностей, програмна система повинна забезпечувати однаковий механізм запуску алгоритмів для різних початкових умов, типів систем і режимів постобробки, зокрема SHA-256. Такий підхід дозволяє використовувати єдину програмну основу як для прикладного шифрування, так і для підготовки даних до статистичного тестування.

Перевагою обраної архітектури є її модульність. Заміна або додавання нового хаотичного генератора не потребує зміни логіки клієнтського інтерфейсу, якщо зберігається загальний формат параметрів і результатів. Аналогічно, розширення переліку підтримуваних типів даних може бути реалізоване через додавання нового конвертера та відповідного режиму криптографічної обробки.

Крім того, винесення обчислень на серверну сторону спрощує контроль коректності реалізації та зменшує навантаження на браузер користувача.

Отже, запропонована архітектура відповідає задачам дипломної роботи, оскільки поєднує інтерфейс для демонстрації шифрування, серверну реалізацію гібридних генераторів, модулі конвертації даних і криптографічні алгоритми для різних типів інформації. Її структура забезпечує відтворюваність результатів, можливість подальшого тестування генераторів та достатню гнучкість для розширення програмного забезпечення.

3.3.2 Опис графічного інтерфейсу

Клієнтська частина програмного застосунку реалізована як односторінковий вебінтерфейс, призначений для демонстрації шифрування та розшифрування тексту, зображень, WAV-аудіофайлів і файлів загального призначення. Основні елементи сторінки згруповано у верхній панелі налаштувань та робочій області для введення або завантаження даних (див. рис. 3.3).

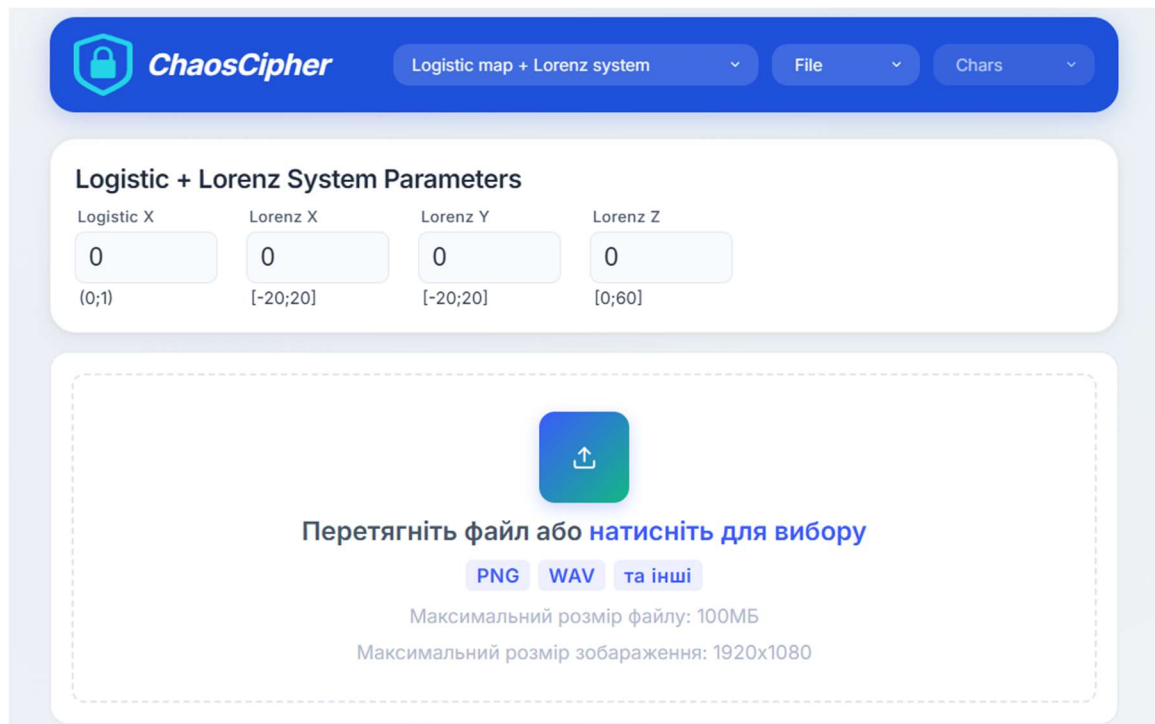


Рисунок 3.3 – Головна сторінка клієнтської частини застосунку

У верхній частині інтерфейсу користувач обирає гібридну хаотичну систему, на основі якої формується псевдовипадкова послідовність для криптографічного перетворення (див. рис. 3.4).

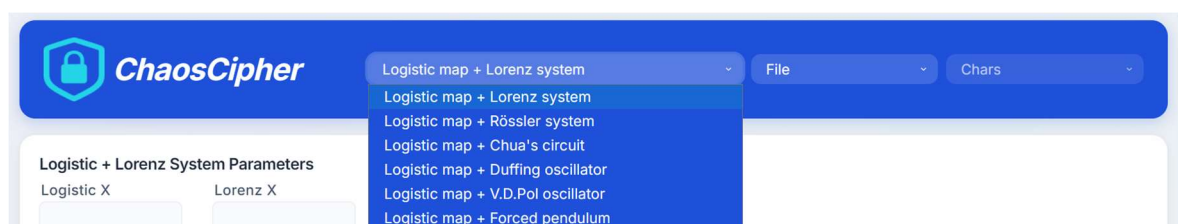


Рисунок 3.4 – Вибір гібридної хаотичної системи

Після вибору системи відображається відповідний набір полів початкових умов. Ці значення виконують роль ключових параметрів генератора і передаються на сервер разом із даними, що підлягають обробці (див. рис. 3.5).

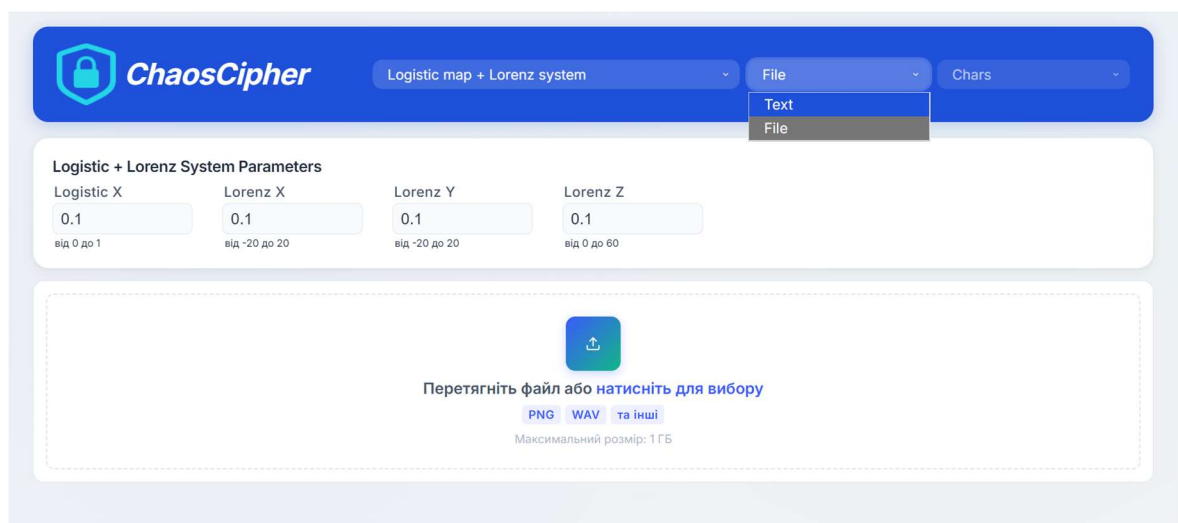


Logistic + Lorenz System Parameters

Logistic X	Lorenz X	Lorenz Y	Lorenz Z
0.1	0	0	0
від 0 до 1	від -20 до 20	від -20 до 20	від 0 до 60

Рисунок 3.5 – Поля введення параметрів обраної хаотичної системи

Окремий випадний список використовується для вибору основного режиму роботи: Text або File (див. рис. 3.6). Залежно від вибраного режиму відкривається відповідний блок робочої області, що спрощує взаємодію користувача із застосунком і не перевантажує сторінку зайвими елементами. В даному випадку обрано режим роботи з файлами, відображається Drop Zone для завантаження файлів.



ChaosCipher Logistic map + Lorenz system File Chars

Text
File

Logistic + Lorenz System Parameters

Logistic X	Lorenz X	Lorenz Y	Lorenz Z
0.1	0.1	0.1	0.1
від 0 до 1	від -20 до 20	від -20 до 20	від 0 до 60

Перетягніть файл або натисніть для вибору.

PNG WAV та інші

Максимальний розмір: 1 ГБ

Рисунок 3.6 – Вибір типу даних для обробки

Користувач може натиснути на цю область і вибрати файл через стандартне вікно операційної системи або перетягнути файл безпосередньо у вебсторінку. Після завантаження файлу система автоматично визначає його підтип: зображення, WAV-аудіофайл або файл загального призначення та відкриває блок, що відповідає типу вхідних даних.

У текстовому режимі інтерфейс містить поле для введення відкритого тексту або шифротексту та поле для виведення результату. Для текстових даних передбачено два режими обробки: побайтовий, що використовує операцію XOR, і посимвольний, що використовує афінне перетворення. Вибір режиму доступний лише для тексту (див. рис. 3.7), в інших випадках він заблокований. Між двома текстовими блоками розташована стрілка, при натисканні на яку вміст полів змінюється місцями. Під текстовими блоками знаходяться кнопки Encrypt та Decrypt. Кнопка Encrypt використовується для шифрування інформації, а кнопка Decrypt – для розшифрування даних. У текстовому режимі повідомлення про помилки введення або обробки можуть відображатися безпосередньо в одному з текстових полів.

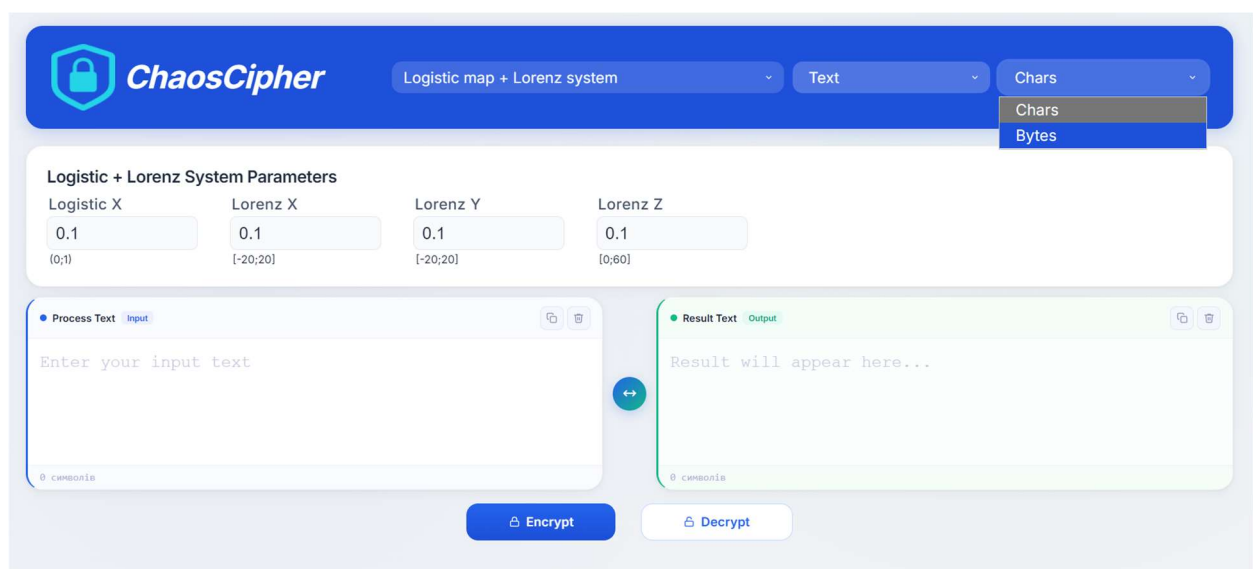


Рисунок 3.7 – Блок роботи з текстовими даними

Кожен текстовий блок містить дві кнопки у правому верхньому куті (див. рис. 3.8). Перша кнопка призначена для копіювання даних до буфера обміну, друга — для очищення текстового поля.

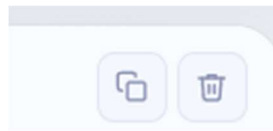


Рисунок 3.8 – Кнопки керування текстовими полями

Блок обробки зображень відкривається для графічних файлів (див. рис. 3.9). Він містить зліва направо: область перегляду початкового зображення, область результату та набір кнопок Encrypt, Decrypt, Download і Clear.

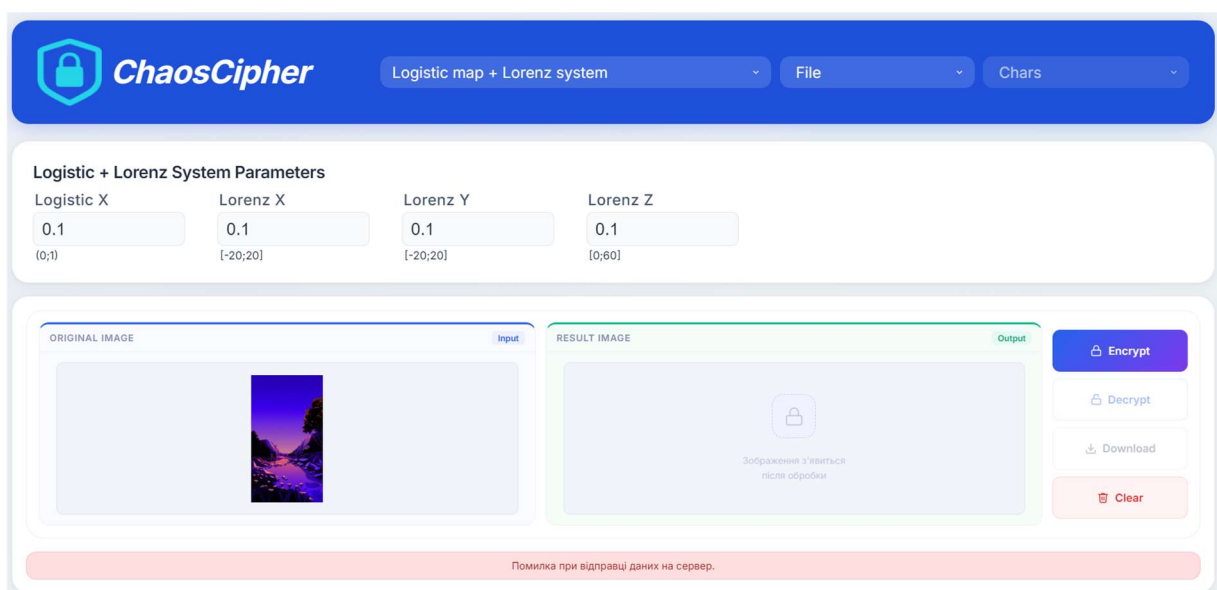


Рисунок 3.9 – Блок обробки зображень

Кнопка Encrypt призначена для шифрування незашифрованих файлів і є недоступною в режимі розшифрування. Кнопка Decrypt запускає процес розшифрування та доступна лише при завантаженні зашифрованих файлів. Кнопка Download використовується для завантаження результату на комп'ютер користувача та доступна лише після обробки оригінального файлу. Кнопка Clear використовується для очищення блоку та повернення до блоку Drop Zone.

Після виконання операції Encrypt або Decrypt користувач може переглянути отримане зображення в області результату та завантажити його як окремий файл (див. рис. 3.10).

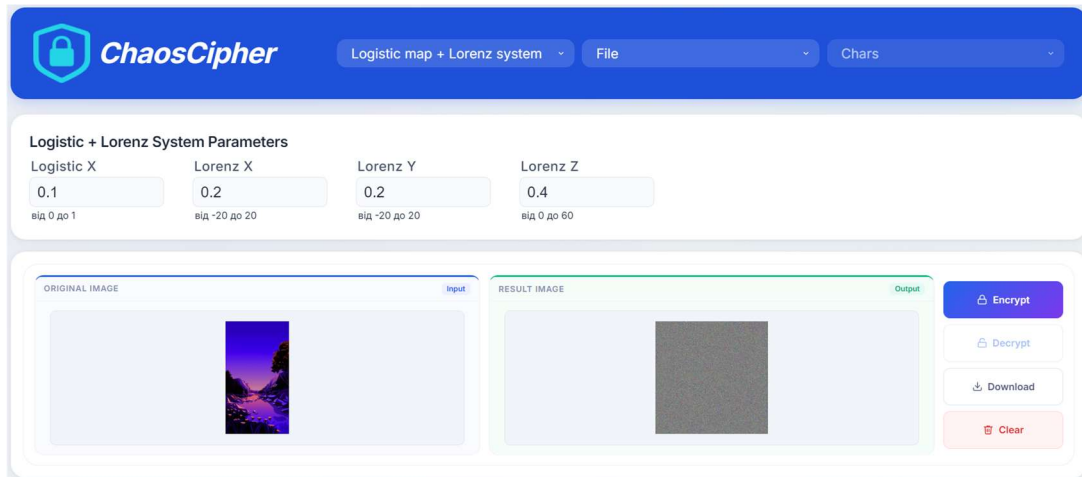


Рисунок 3.10 – Доступна кнопка Download після обробки зображення

Для WAV-аудіофайлів передбачено окремий блок, що містить аудіоплеєр для початкового файлу, секцію для результуючих даних, яка відображається після обробки, а також кнопки Encrypt, Decrypt, Download і Clear. Завдяки аудіоплеєру користувач може прослухати вхідний або оброблений файл безпосередньо у браузері (див. рис. 3.11).

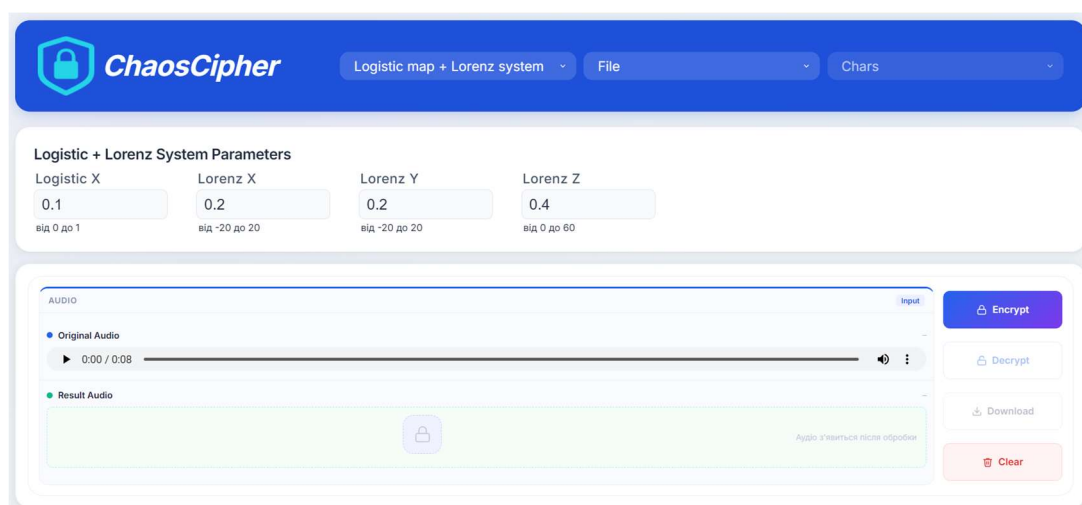


Рисунок 3.11 – Блок обробки WAV-аудіофайлів

Для інших типів файлів використовується універсальний файловий блок, де файл розглядається як байтова послідовність без аналізу його внутрішньої структури. Інтерфейс та функціональний набір кнопок блоку для інших типів файлів подібний до блоку обробки зображень (див. рис. 3.12).

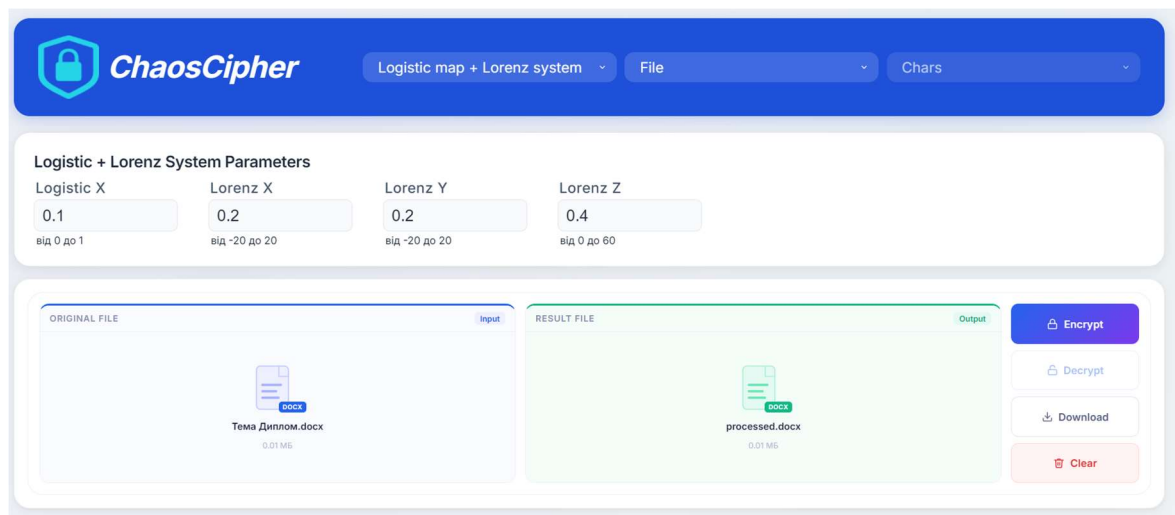


Рисунок 3.12 – Блок обробки файлів загального призначення

Таким чином, основні дії користувача виконуються кнопками Encrypt та Decrypt. Під час натискання відповідної кнопки клієнтська частина формує HTTP-запит до серверної частини, реалізованої з використанням фреймворку FastAPI, передаючи тип операції, тип даних, обрану хаотичну систему, початкові умови та вхідні дані. Сервер виконує генерацію псевдовипадкової послідовності, здійснює шифрування або розшифрування та повертає результат у клієнтський інтерфейс.

Крім локального відображення помилок у текстовому режимі, інтерфейс містить загальний інформаційний блок для виведення системних повідомлень про некоректне введення параметрів, неправильний формат файла або помилки обробки (див. рис. 3.13). Такий підхід дає змогу користувачу швидко визначити причину помилки, наприклад неправильний формат файла, невідповідні початкові умови або спробу розшифрування з неправильними параметрами.

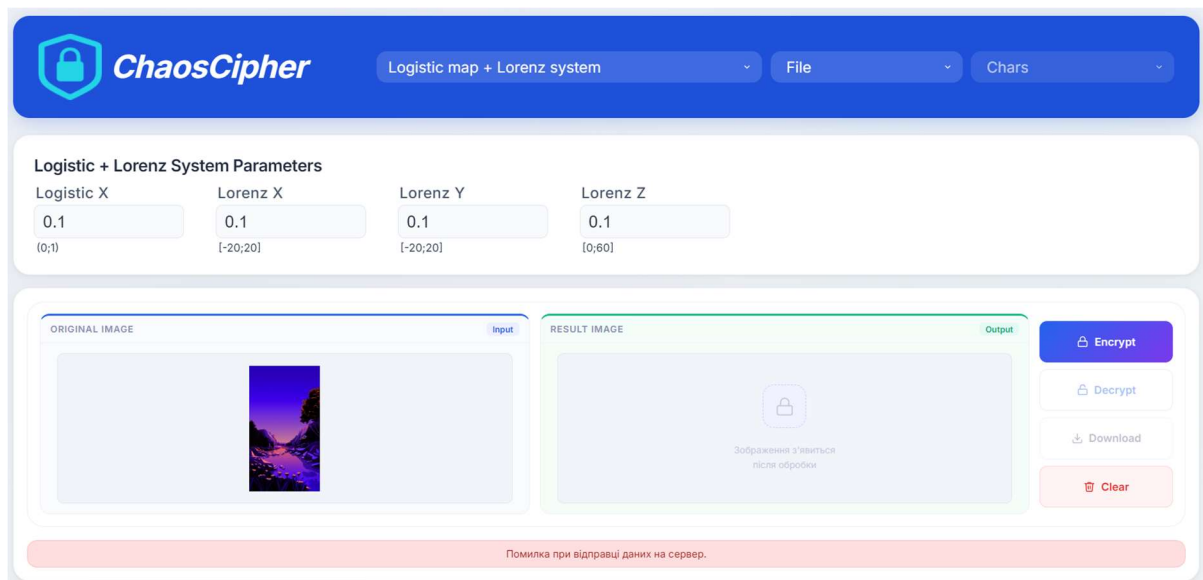


Рисунок 3.13 – Повідомлення помилки обробки

Отже, графічний інтерфейс застосунку забезпечує повний цикл роботи з даними: вибір хаотичної системи, введення параметрів, вибір типу даних, запуск шифрування або розшифрування, перегляд результату та його збереження. Структура інтерфейсу є достатньо простою для демонстраційного використання, водночас вона охоплює всі функції, необхідні для демонстрації та перевірки роботи реалізованих алгоритмів.

3.4 Тестування працездатності демонстраційного застосунку

Для перевірки працездатності, стабільності та коректності функціонування програмного забезпечення було розроблено окремі Python-скрипти з юніт-тестами. Їх використання дало змогу автоматизовано перевірити правильність роботи ключових компонентів бекенду додатку, виявити можливі помилки на ранніх етапах тестування, а також оцінити надійність взаємодії між окремими модулями системи. За допомогою даних скриптів було протестовано основні підпрограми бекенду: генератори псевдохаотичної послідовності, модулі шифрування зображень, аудіо та відео, а також API-ендпойнти, що забезпечують взаємодію клієнтської та серверної частин системи.

Для кожного з генераторів псевдовипадкової послідовності було проведено наступний ряд тестів:

1. Перевірка детермінованості генератора – тестує властивість генератора створювати однакові послідовності при повторних викликах з однаковими початковими параметрами.

2. Перевірка відповідності довжини згенерованої послідовності – тестує чи дорівнює фактична довжина результату запитаній кількості елементів.

3. Перевірка оновлення стану генератора – тестує зміну внутрішнього стану хаотичної системи після виконання генерації.

4. Перевірка чутливості до початкових умов – тестує одну з основних властивостей хаотичних систем, а саме значну зміну результатів при мінімальній зміні початкових параметрів.

5. Перевірка стійкості до переповнення та некоректних значень – тестує стабільність роботи генератора під час генерації послідовності довжиною 100000 елементів.

6. Перевірка відсутності циклічних повторень – тестує здатність генератора уникати переходу в короткі цикли повторення значень протягом великої кількості ітерацій.

7. Перевірка належності значень логістичного відображення до інтервалу $(0;1)$ – тестує чи залишаються всі значення логістичного відображення в допустимому інтервалі під час багатьох ітерацій.

8. Перевірка обробки некоректного значення $\text{logistic_x} = 0$ – тестує механізм валідації параметрів логістичного відображення на виникнення помилки при передачі значення, яке призводить до виродження генератора.

9. Перевірка обробки некоректного значення $\text{logistic_x} = 1$ – тестує спроможність системи виявляти недопустиме значення.

Успішні результати тестування для кожного генератору показано на рисунка 3.14 та 3.15

```

✓ Tests passed: 39 of 39 tests – 2 sec 950 ms

C:\Users\sasha\Desktop\lab10\23pythonProject1\venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python
Testing started at 23:49 ...
Launching pytest with arguments C:\Users\sasha\Desktop\lab10\23pythonProject1\venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python
===== test session starts =====
collecting ... collected 39 items

test_chaos_systems.py::TestChaosSystemCommon::test_determinism[lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_determinism[rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_determinism[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_determinism[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_determinism[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_determinism[forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_sequence_length[500-lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_sequence_length[500-rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_sequence_length[500-chua]
test_chaos_systems.py::TestChaosSystemCommon::test_sequence_length[500-duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_sequence_length[500-van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_sequence_length[500-forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_state_updated[lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_state_updated[rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_state_updated[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_state_updated[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_state_updated[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_state_updated[forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[lorenz]

```

Рисунок 3.14 – Результати юніт-тестування модулів генераторів

```

test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_sensitivity_to_initial_conditions[forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_finite_values[forced_pendulum]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[lorenz]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[rossler]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[chua]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[duffing]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[van_der_pol]
test_chaos_systems.py::TestChaosSystemCommon::test_dont_fall_into_circles[forced_pendulum]
test_chaos_systems.py::TestLogisticMap::test_stays_in_unit_interval PASSED [ 2%] PASSED [ 5%] PASSED [ 7%] PASSED [ 10%]
test_chaos_systems.py::TestLogisticMap::test_invalid_logistic_x_zero
test_chaos_systems.py::TestLogisticMap::test_invalid_logistic_x_one

===== 39 passed in 3.05s =====
PASSED [ 94%] PASSED [ 97%] PASSED [100%]
Process finished with exit code 0

```

Рисунок 3.15 – Результати юніт-тестування модулів генераторів

Після перевірки коректності роботи генераторів псевдохаотичних послідовностей було проведено тестування модулів шифрування даних, які використовують згенеровані послідовності як основу для криптографічних перетворень.

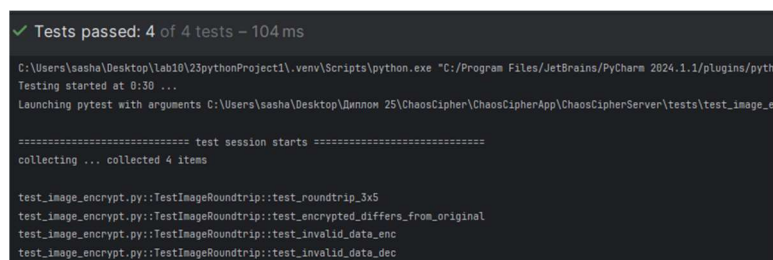
Для модулів шифрування файлів різних типів (зображень, аудіо та відео) було виконано такі тести:

1. Перевірка цілісності циклу “шифрування–розшифрування” аудіофайлу – тестує можливість коректного відновлення аудіоданих після їх шифрування та подальшого розшифрування. Перевіряється, чи збігаються початкові аудіодані з результатом розшифрування при використанні однакових параметрів генератора.

2. Перевірка відмінності зашифрованих даних від початкових – тестує ефективність процесу шифрування. Перевіряється, що зашифрований файл відрізняється від оригінального файлу та не зберігає вихідні дані у відкритому вигляді.

3. Перевірка обробки некоректних даних при розшифруванні – тестує стійкість алгоритму до некоректних або відсутніх вхідних даних. Перевіряється, чи виникає відповідне виключення при передачі невалідного значення замість файлу.

4. Перевірка обробки помилкових даних при розшифруванні незашифрованого файлу – тестує механізм виявлення неправильного формату або структури даних під час розшифрування. Перевіряється, чи повертається помилка при спробі розшифрувати дані, які не були попередньо зашифровані. Результати відображені на рис 3.16, 3.17 та 3.18



```
✓ Tests passed: 4 of 4 tests - 104 ms

C:\Users\sasha\Desktop\lab10\23pythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python
Testing started at 0:30 ...
Launching pytest with arguments C:\Users\sasha\Desktop\Дивном 25\ChaosCipher\ChaosCipherApp\ChaosCipherServer\tests\test_image_enc
===== test session starts =====
collecting ... collected 4 items

test_image_encrypt.py::TestImageRoundtrip::test_roundtrip_3x5
test_image_encrypt.py::TestImageRoundtrip::test_encrypted_differs_from_original
test_image_encrypt.py::TestImageRoundtrip::test_invalid_data_enc
test_image_encrypt.py::TestImageRoundtrip::test_invalid_data_dec
```

Рисунок 3.16 – Результати юніт-тестування шифратора зображення

```
✓ Tests passed: 4 of 4 tests – 3 ms

C:\Users\sasha\Desktop\lab10\23pythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python
Testing started at 0:28 ...
Launching pytest with arguments test_audio_encrypt.py::TestAudioRoundtrip --no-header --no-summary -q in C:\Users\sasha\Desktop\lab10\23pythonProject1
===== test session starts =====
collecting ... collected 4 items

test_audio_encrypt.py::TestAudioRoundtrip::test_roundtrip_wav PASSED [ 25%]
test_audio_encrypt.py::TestAudioRoundtrip::test_encrypted_differs PASSED [ 50%]
test_audio_encrypt.py::TestAudioRoundtrip::test_invalid_data_enc PASSED [ 75%]
test_audio_encrypt.py::TestAudioRoundtrip::test_invalid_data_dec PASSED [100%]

===== 4 passed in 0.06s =====

Process finished with exit code 0
```

Рисунок 3.17 – Результати юніт-тестування шифратора аудіо

```
✓ Tests passed: 4 of 4 tests – 0 ms

C:\Users\sasha\Desktop\lab10\23pythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python
Testing started at 0:33 ...
Launching pytest with arguments test_file_encrypt.py::TestFileRoundtrip --no-header --no-summary -q in C:\Users\sasha\Desktop\lab10\23pythonProject1
===== test session starts =====
collecting ... collected 4 items

test_file_encrypt.py::TestFileRoundtrip::test_roundtrip[all_byte_values] PASSED [ 25%]
test_file_encrypt.py::TestFileRoundtrip::test_invalid_data_enc PASSED [ 50%]
test_file_encrypt.py::TestFileRoundtrip::test_invalid_data_dec PASSED [ 75%]
test_file_encrypt.py::TestFileRoundtrip::test_encrypted_differs PASSED [100%]

===== 4 passed in 0.03s =====

Process finished with exit code 0
```

Рисунок 3.18 – Результати юніт-тестування шифратора відео

Окрім шифрування мультимедійних та бінарних файлів, окрему увагу було приділено перевірці механізмів шифрування текстової інформації. З цією метою було виконано наступний набір тестів:

1. Перевірка коректності шифрування та розшифрування тексту в символьному режимі – тестує здатність системи правильно відновлювати початковий текст різної довжини та різних алфавітів після шифрування та розшифрування у режимі роботи з символами.

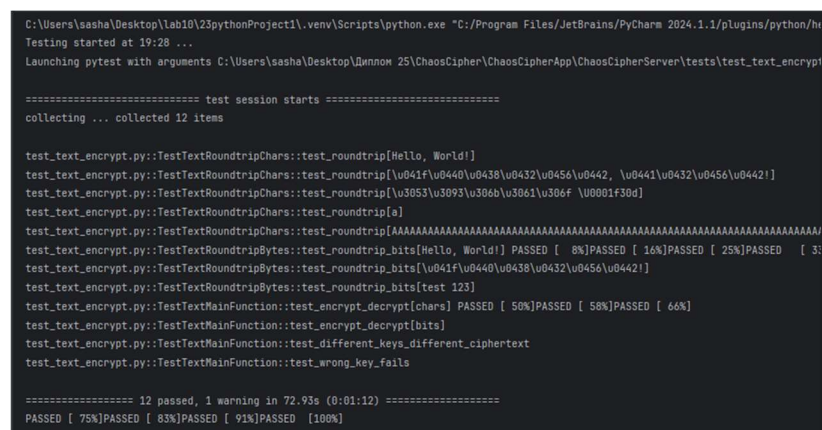
2. Перевірка коректності шифрування та розшифрування тексту в байтовому режимі – тестує правильність роботи алгоритму у режимі побайтової обробки даних. Перевіряється, чи співпадає розшифрований текст із початковим повідомленням після виконання циклу шифрування та розшифрування.

3. Перевірка універсальної функції шифрування та розшифрування – тестує коректність роботи основних функцій `encrypt_text()` та `decrypt_text()` у двох режимах роботи: символьному та байтовому.

4. Перевірка залежності шифротексту від ключа – тестує властивість криптографічної системи генерувати різні шифротексти для одного й того самого відкритого тексту при використанні різних хаотичних систем та параметрів генератора.

5. Перевірка стійкості до використання неправильного ключа – тестує неможливість коректного відновлення початкового повідомлення при розшифруванні з використанням іншого набору параметрів або іншої хаотичної системи.

Результати юніт-тестування шифратора тексту наведено на рисунку 3.19.



```
C:\Users\sasha\Desktop\lab10\23pythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python/h...
Testing started at 19:28 ...
Launching pytest with arguments C:\Users\sasha\Desktop\lab10\23pythonProject1\ChaosCipherApp\ChaosCipherServer\tests\test_text_encrypt.py
===== test session starts =====
collecting ... collected 12 items

test_text_encrypt.py::TestTextRoundtripChars::test_roundtrip[Hello, World!]
test_text_encrypt.py::TestTextRoundtripChars::test_roundtrip[\u041f\u0440\u0438\u0456\u0442, \u0441\u0432\u0442\u0442!]
test_text_encrypt.py::TestTextRoundtripChars::test_roundtrip[\u0305\u0303\u0306\u0306f \u0001f30d]
test_text_encrypt.py::TestTextRoundtripChars::test_roundtrip[a]
test_text_encrypt.py::TestTextRoundtripChars::test_roundtrip[AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA]
test_text_encrypt.py::TestTextRoundtripBytes::test_roundtrip_bits[Hello, World!] PASSED [ 8%]PASSED [ 16%]PASSED [ 25%]PASSED [ 33%]
test_text_encrypt.py::TestTextRoundtripBytes::test_roundtrip_bits[\u041f\u0440\u0438\u0456\u0442\u0442!]
test_text_encrypt.py::TestTextRoundtripBytes::test_roundtrip_bits[test 123]
test_text_encrypt.py::TestTextMainFunction::test_encrypt_decrypt[chars] PASSED [ 50%]PASSED [ 58%]PASSED [ 66%]
test_text_encrypt.py::TestTextMainFunction::test_encrypt_decrypt[bits]
test_text_encrypt.py::TestTextMainFunction::test_different_keys_different_ciphertext
test_text_encrypt.py::TestTextMainFunction::test_wrong_key_fails

===== 12 passed, 1 warning in 72.93s (0:01:12) =====
PASSED [ 75%]PASSED [ 83%]PASSED [ 91%]PASSED [ 100%]
```

Рисунок 3.19 – Результати юніт-тестування шифратора тексту

Після перевірки внутрішньої логіки роботи алгоритмів було виконано тестування API-ендпойнтів, які забезпечують взаємодію між клієнтською та серверною частинами системи.

Для ендпойнтів було проведено тестування на:

1. Перевірка коректності шифрування тексту – тестує успішність виконання запиту, коректність HTTP-відповіді та наявність зашифрованого результату у відповіді сервера.

2. Перевірка цілісності циклу “шифрування–розшифрування” для тексту – тестує можливість повного відновлення початкового тексту після шифрування та подальшого розшифрування.

3. Перевірка коректності шифрування зображень – тестує успішність обробки файлу та відповідність типу отриманих даних формату зображення.

4. Перевірка цілісності циклу “шифрування–розшифрування” для зображень – тестує можливість коректного відновлення зображення після виконання операцій шифрування та розшифрування.

5. Перевірка коректності шифрування аудіофайлів – тестує можливість обробки та шифрування аудіоданих через API.

6. Перевірка коректності шифрування довільних файлів – тестує успішність обробки бінарного файлу та правильність формування відповіді сервера.

7. Перевірка обробки невідомої системи генерації – тестує механізм валідації параметрів системи шифрування.

8. Перевірка обробки некоректних параметрів JSON – тестує стійкість API до помилкових параметрів конфігурації. Перевіряється, чи коректно сервер обробляє невалідний JSON та повертає відповідний код помилки.

Результати юніт-тестування API-ендпойнтів наведено на рисунку 3.20.

```
✓ Tests passed: 9 of 9 tests – 1 sec 530 ms
C:\Users\sasha\Desktop\lab10\23pythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2024.1.1/plugins/python
Testing started at 1:50 ...
Launching pytest with arguments C:\Users\sasha\Desktop\Думном 25\ChaosCipher\ChaosCipherApp\ChaosCipherServer\tests\test_api_endpo

===== test session starts =====
collecting ... collected 9 items

test_api_endpoints.py::TestTextEndpoints::test_encrypt_text
test_api_endpoints.py::TestTextEndpoints::test_encrypt_decrypt_text_roundtrip
test_api_endpoints.py::TestImageEndpoints::test_encrypt_image PASSED [ 11%]logisticLorenz: 0.29999999999999999
lorenzX: 1
lorenzY: 1
lorenzZ: 1
chars
PASSED [ 22%]logisticLorenz: 0.29999999999999999
lorenzX: 1
lorenzY: 1
lorenzZ: 1
chars
logisticLorenz: 0.29999999999999999
lorenzX: 1
lorenzY: 1
lorenzZ: 1

test_api_endpoints.py::TestImageEndpoints::test_decrypt_unencrypted_image_throws_error
test_api_endpoints.py::TestImageEndpoints::test_encrypt_decrypt_image_roundtrip
test_api_endpoints.py::TestAudioEndpoints::test_encrypt_audio PASSED [ 33%]logisticLorenz: 0.29999999999999999
lorenzX: 1
lorenzY: 1
lorenzZ: 1
```

Рисунок 3.20 – Результати юніт-тестування ендпойнтів

Отже, відповідно до представлених ілюстрацій, усі тести ключових модулів програмної системи завершилися успішно, що підтверджує коректність роботи алгоритмів, стійкість до помилкових вхідних даних та надійність функціонування системи. Результати тестування демонструють стабільність генераторів псевдовипадкових послідовностей, коректність процесів шифрування та розшифрування, а також здатність системи обробляти різні типи даних без втрати цілісності інформації.

Висновки до розділу 3

У цьому розділі розглянуто програмну реалізацію демонстраційного веб-додатку для шифрування та розшифрування даних на основі розроблених гібридних хаотичних генераторів. Обґрунтовано вибір засобів розробки, зокрема Python для реалізації математичної та криптографічної логіки, FastAPI для серверної частини, JavaScript для взаємодії з інтерфейсом користувача, а також допоміжних бібліотек для обробки різних типів даних.

15

Було визначено функціональні та нефункціональні вимоги до системи, описано основні сценарії використання, вхідні дані та архітектуру застосунку. Програмне рішення побудовано за клієнт-серверною моделлю та має модульну структуру, що включає клієнтський інтерфейс, серверну частину, модулі конвертації даних, генератор хаотичних послідовностей і криптографічний модуль.

Розроблений застосунок забезпечує шифрування та розшифрування тексту, зображень, аудіофайлів і файлів загального призначення, а також підтримує вибір параметрів генератора, попередній перегляд і збереження результатів. Проведене тестування підтвердило коректність роботи основних модулів, відтворюваність результатів, чутливість до зміни початкових параметрів і стійкість до некоректних вхідних даних. Таким чином, створений веб-додаток підтвердив практичну придатність запропонованих гібридних хаотичних генераторів для реалізації процедур шифрування та розшифрування інформації різних типів.

ВИСНОВКИ

У кваліфікаційній роботі на тему «Дослідження ефективності застосування гібридних хаотичних систем для шифрування даних» проведено дослідження можливостей використання гібридних детермінованих хаотичних систем для формування псевдовипадкових послідовностей і шифрування даних різних типів. У межах роботи виконано теоретичний аналіз хаотичних систем, розроблено гібридні генератори на основі логістичного відображення та неперервних систем, проведено статистичне тестування, оцінено швидкодію та реалізовано програмний веб-додаток для шифрування і розшифрування даних.

Теоретична гіпотеза роботи полягала в тому, що поєднання дискретного логістичного відображення з неперервними хаотичними системами може забезпечити формування статистично якісних ключових послідовностей, придатних для шифрування даних. За результатами дослідження ця гіпотеза загалом підтвердилася: усі розроблені гібридні генератори продемонстрували високі результати статистичного тестування NIST STS та практичну придатність для криптографічного перетворення даних. Водночас експериментальні результати показали, що ефективність генератора визначається не лише інтенсивністю хаосу базової системи, а й способом гібридизації, режимом формування бітової послідовності, використанням SHA-256, вибором початкових станів і особливостями програмної реалізації.

За результатами теоретичного аналізу встановлено, що ефективність хаотичних систем у задачах шифрування залежить від поєднання інтенсивності хаосу, складності атратора та ресурсоємності реалізації. Найвищу інтенсивність хаосу серед неперервних систем показала система Лоренца, тоді як логістичне відображення виявилось найбільш ефективним дискретним компонентом завдяки простоті реалізації та високій хаотичності. Система Чуа продемонструвала збалансовані характеристики, а осцилятор Дуффінга, Ван дер Поля, Forced Pendulum і система Ресслера мають окремі переваги та обмеження, пов'язані з рівнем хаосу або обчислювальними витратами.

На основі проведеного аналізу сформовано шість гібридних генераторів: Logistic-Lorenz, Logistic-Rossler, Logistic-Chua, Logistic-Duffing, Logistic-Van der Pol та Logistic-Forced Pendulum. Їхня робота ґрунтується на поєднанні логістичного відображення з неперервною хаотичною системою, стан якої моделюється методом Рунге–Кутта четвертого порядку. Такий підхід дозволяє поєднати простоту дискретного хаосу з багатовимірною динамікою неперервних систем і сформувати складний ключовий потік для шифрування.

Статистичне тестування за допомогою NIST STS у режимах Float та SHA-256 для одного й декількох початкових станів підтвердило високу якість згенерованих послідовностей. Найкращий інтегральний показник Float/SHA-256/OS/MS отримано для Logistic-Chua — 0,998; далі йдуть Logistic-Duffing — 0,996 та Logistic-Lorenz — 0,994. Це підтверджує теоретичні очікування щодо перспективності систем Чуа та Лоренца. Водночас високі результати Logistic-Duffing і Logistic-Forced Pendulum показали, що практична ефективність може бути високою навіть для систем, які теоретично не є лідерами за інтенсивністю хаосу, але мають складну динаміку та добре працюють у гібридній реалізації.

Оцінювання швидкодії показало, що режим SHA-256 у реалізованому програмному середовищі є швидшим за Float для всіх генераторів. Найвищу швидкість продемонстрував Logistic-Forced Pendulum — 4,105 MB/s, близькі результати отримано для Logistic-Duffing — 4,073 MB/s та Logistic-Lorenz — 3,922 MB/s. За комплексним індексом, який враховує статистичну якість і швидкодію, найкращими виявилися Logistic-Forced Pendulum та Logistic-Duffing. Отже, теоретична гіпотеза підтвердилася не повністю лінійно: найбільша інтенсивність хаосу не завжди прямо відповідає найкращому комплексному результату генератора.

Порівняння з традиційними генераторами BBS, Modular Exponentiation та Micali–Schnorr показало, що класичні генератори мають сильніше теоретичне обґрунтування криптостійкості, але є більш ресурсоємними через використання операцій над великими числами. Гібридні хаотичні генератори, хоча й потребують подальшого формального криптоаналізу, демонструють високі

статистичні показники, кращу гнучкість налаштування та практичну швидкодію, що підтверджує доцільність їх застосування у задачах шифрування даних.

Практична реалізація підтвердила можливість застосування розроблених генераторів для шифрування тексту, зображень, аудіофайлів і файлів загального призначення. Розроблений веб-додаток на основі Python, FastAPI та JavaScript забезпечує вибір типу даних, режиму роботи, хаотичної системи та початкових параметрів генератора, а також коректне виконання процесів шифрування і розшифрування.

Отримані результати значною мірою узгоджуються з попереднім теоретичним аналізом, де Lorenz, Chua і Duffing були виділені як системи з високою динамічною значущістю: Lorenz — за інтенсивністю хаосу, Chua — за поєднанням хаотичності та фрактальної складності, Duffing — за складною структурою атрактора. Водночас інші системи також продемонстрували достатньо якісні характеристики генераторів, що підтверджує перспективність гібридних хаотичних систем для криптографічного захисту інформації